

Arquitetura de LLMs e IA Aplicada Do Basico ao Avancado com RAG Corporativo

Autores do Projeto

2 de maio de 2026

Prefacio

Este livro propoe uma trilha pratica e incremental para construir sistemas com LLMs e IA, com foco em uso corporativo, RAG, avaliacao continua e governanca tecnica.

Como ler este livro

- Cada capitulo possui teoria, implementacao guiada e plano de testes.
- A complexidade evolui do basico para o avancado.
- O objetivo e manter rastreabilidade entre conceito, codigo e evidencias.

Indice proposto

1. Fundamentos de IA e LLMs
2. Ambiente, ferramentas e reprodutibilidade
3. Agentes com LangGraph e memoria
4. Tools, skills e MCP
5. Fundamentos de RAG
6. RAG corporativo e arquitetura de dados
7. Pipeline assincrono em AWS
8. Segmentacao, personas e digital twins
9. GraphRAG com Neptune
10. Confiabilidade, observabilidade e operacao
11. Metricas de avaliacao de mercado
12. LLM as Judge
13. Golden dataset e regressao de qualidade
14. Roadmap de producao e evolucao continua

Sumário

Prefacio	i
1 Fundamentos de IA e Modelos de Linguagem de Grande Escala	1
1.1 Escopo do capítulo	1
1.2 Objetivos de aprendizagem	1
1.3 Conceitos teóricos	2
1.3.1 Arquitetura Transformer	2
1.3.2 Tokenização	2
1.3.3 Janela de Contexto	3
1.3.4 Parâmetros de Amostragem	3
1.4 Implementação orientada por passos	3
1.4.1 Passo 1 — Configuração das credenciais AWS	3
1.4.2 Passo 2 — Primeira chamada ao modelo	4
1.4.3 Passo 3 — Engenharia de <i>prompts</i>	5
1.4.4 Passo 4 — Técnica <i>few-shot</i>	6
1.5 Plano de testes do capítulo	6
1.5.1 Teste funcional	6
1.5.2 Teste de desempenho	6
1.5.3 Teste de qualidade de resposta	6
1.6 Critérios de aceite	7
1.7 Espaço para expansão iterativa	7
2 Ambiente, Ferramentas e Reprodutibilidade	8
2.1 Escopo do capítulo	8
2.2 Objetivos de aprendizagem	8
2.3 Conceitos e prerequisites	9
2.3.1 Versões e dependências	9
2.3.2 Estrutura do repositório	9
2.4 Implementação orientada por passos	10
2.4.1 Passo 1 — Clonar o repositório e criar o ambiente virtual	10
2.4.2 Passo 2 — Configurar credenciais AWS	10

2.4.3	Passo 3 — Provisionar a infraestrutura com Terraform	11
2.4.4	Passo 4 — Executar os notebooks	11
2.5	Plano de testes do capítulo	12
2.5.1	Teste funcional	12
2.5.2	Teste de desempenho	12
2.5.3	Teste de qualidade	12
2.6	Critérios de aceite	12
2.7	Espaço para expansão iterativa	13
3	Agentes com LangGraph e Memória	14
3.1	Escopo do capítulo	14
3.2	Objetivos de aprendizagem	14
3.3	Conceitos teóricos	15
3.3.1	Grafo de estados e nós de processamento	15
3.3.2	Ciclo ReAct	15
3.3.3	Persistência de estado com MemorySaver	15
3.4	Implementação orientada por passos	16
3.4.1	Passo 1 — Definir as ferramentas do agente	16
3.4.2	Passo 2 — Construir o grafo ReAct	16
3.4.3	Passo 3 — Invocar o agente com persistência de contexto	17
3.4.4	Passo 4 — Integrar memória vetorial FAISS	18
3.5	Plano de testes do capítulo	19
3.5.1	Teste funcional	19
3.5.2	Teste de desempenho	19
3.5.3	Teste de qualidade de resposta	19
3.6	Critérios de aceite	19
3.7	Espaço para expansão iterativa	20
4	Tools, Skills e MCP	21
4.1	Escopo do capítulo	21
4.2	Objetivos de aprendizagem	21
4.3	Implementação orientada por passos	21
4.4	Plano de testes do capítulo	21
4.4.1	Teste funcional	21
4.4.2	Teste de desempenho	22
4.4.3	Teste de qualidade de resposta	22
4.5	Criterios de aceite	22
4.6	Espaco para expansao iterativa	22

5	Fundamentos de RAG	23
5.1	Escopo do capítulo	23
5.2	Objetivos de aprendizagem	23
5.3	Implementação orientada por passos	23
5.4	Plano de testes do capítulo	23
5.4.1	Teste funcional	23
5.4.2	Teste de desempenho	24
5.4.3	Teste de qualidade de resposta	24
5.5	Critérios de aceite	24
5.6	Espaço para expansão iterativa	24
6	RAG Corporativo e Arquitetura de Dados	25
6.1	Escopo do capítulo	25
6.2	Objetivos de aprendizagem	25
6.3	Conceitos teóricos	26
6.3.1	RAG em ambiente regulado	26
6.3.2	Tipos de documento no projeto	26
6.4	Implementação orientada por passos	26
6.4.1	Passo 1 — Preparar coleção de documentos	26
6.4.2	Passo 2 — Ingerir texto no endpoint <code>/documentos</code>	27
6.4.3	Passo 3 — Verificar chunking e indexação	27
6.4.4	Passo 4 — Consumir o contexto em consulta real	28
6.4.5	Passo 5 — Resultado completo da aula	28
6.5	Plano de testes do capítulo	28
6.5.1	Teste funcional	28
6.5.2	Teste de desempenho	29
6.5.3	Teste de qualidade de resposta	29
6.6	Critérios de aceite	29
6.7	Espaço para expansão iterativa	29
7	Pipeline Assíncrono em AWS	30
7.1	Escopo do capítulo	30
7.2	Objetivos de aprendizagem	30
7.3	Conceitos teóricos	30
7.3.1	Padrão fire-and-poll	30
7.3.2	Estados de processamento	31
7.4	Implementação orientada por passos	31
7.4.1	Passo 1 — Submeter consulta e enfileirar	31
7.4.2	Passo 2 — Processar no worker	31
7.4.3	Passo 3 — Consultar status	32

7.4.4	Passo 4 — Teste ponta a ponta de laboratorio	32
7.4.5	Passo 5 — Resultado completo da aula	32
7.4.6	Passo 6 — Evidencia operacional via frontend CloudFront	33
7.5	Plano de testes do capitulo	33
7.5.1	Teste funcional	33
7.5.2	Teste de desempenho	33
7.5.3	Teste de qualidade de resposta	33
7.6	criterios de aceite	34
7.7	Espaco para expansao iterativa	34
8	Segmentacao, Personas e Digital Twins	35
8.1	Escopo do capitulo	35
8.2	Objetivos de aprendizagem	35
8.3	Conceitos teoricos	35
8.3.1	Tres superficies de analise	35
8.3.2	Features de clusterizacao	36
8.4	Implementacao orientada por passos	36
8.4.1	Passo 1 — Gerar base sintetica e executar clustering	36
8.4.2	Passo 2 — Criar contexto RAG por cluster	37
8.4.3	Passo 3 — Instanciar agente persona	37
8.4.4	Passo 4 — Criar digital twin sob demanda	37
8.4.5	Passo 5 — Chamar a API em cada modo	38
8.4.6	Passo 6 — Resultado completo da aula	38
8.4.7	Passo 7 — Evidencias de execucao em producao	38
8.5	Plano de testes do capitulo	39
8.5.1	Teste funcional	39
8.5.2	Teste de desempenho	39
8.5.3	Teste de qualidade de resposta	39
8.6	criterios de aceite	39
8.7	Espaco para expansao iterativa	40
9	GraphRAG com Neptune	41
9.1	Escopo do capitulo	41
9.2	Objetivos de aprendizagem	41
9.3	Conceitos teoricos	41
9.3.1	Do RAG textual ao GraphRAG	41
9.3.2	Fontes de contexto combinadas	42
9.4	Implementacao orientada por passos	42
9.4.1	Passo 1 — Popular o grafo de conhecimento	42
9.4.2	Passo 2 — Ingerir documentos para mapeamento textual	42

9.4.3	Passo 3 — Executar consulta relacional no Neptune	42
9.4.4	Passo 4 — Fundir contexto textual e relacional	43
9.4.5	Passo 5 — Validar em consulta de negocio	43
9.4.6	Passo 6 — Resultado completo da aula	44
9.5	Plano de testes do capitulo	44
9.5.1	Teste funcional	44
9.5.2	Teste de desempenho	44
9.5.3	Teste de qualidade de resposta	44
9.6	criterios de aceite	45
9.7	Espaco para expansao iterativa	45
10	Confiabilidade, Observabilidade e Operacao	46
10.1	Escopo do capitulo	46
10.2	Objetivos de aprendizagem	46
10.3	Implementacao orientada por passos	46
10.4	Plano de testes do capitulo	46
10.4.1	Teste funcional	46
10.4.2	Teste de desempenho	47
10.4.3	Teste de qualidade de resposta	47
10.5	criterios de aceite	47
10.6	Espaco para expansao iterativa	47
11	Metricas de Avaliacao de Mercado	48
11.1	Escopo do capitulo	48
11.2	Objetivos de aprendizagem	48
11.3	Implementacao orientada por passos	48
11.4	Plano de testes do capitulo	48
11.4.1	Teste funcional	48
11.4.2	Teste de desempenho	49
11.4.3	Teste de qualidade de resposta	49
11.5	criterios de aceite	49
11.6	Espaco para expansao iterativa	49
12	LLM as Judge	50
12.1	Escopo do capitulo	50
12.2	Objetivos de aprendizagem	50
12.3	Implementacao orientada por passos	50
12.4	Plano de testes do capitulo	50
12.4.1	Teste funcional	50
12.4.2	Teste de desempenho	51

12.4.3	Teste de qualidade de resposta	51
12.5	Critérios de aceite	51
12.6	Espaco para expansao iterativa	51
13	Golden Dataset e Regressao de Qualidade	52
13.1	Escopo do capitulo	52
13.2	Objetivos de aprendizagem	52
13.3	Implementacao orientada por passos	52
13.4	Plano de testes do capitulo	52
13.4.1	Teste funcional	52
13.4.2	Teste de desempenho	53
13.4.3	Teste de qualidade de resposta	53
13.5	Critérios de aceite	53
13.6	Espaco para expansao iterativa	53
14	Roadmap de Producao e Evolucao Continua	54
14.1	Escopo do capitulo	54
14.2	Objetivos de aprendizagem	54
14.3	Implementacao orientada por passos	54
14.4	Plano de testes do capitulo	54
14.4.1	Teste funcional	54
14.4.2	Teste de desempenho	55
14.4.3	Teste de qualidade de resposta	55
14.5	Critérios de aceite	55
14.6	Espaco para expansao iterativa	55
15	Esteiras de CI/CD Multi-Ambiente para Produtos AWS Bancários	56
15.1	Escopo do capítulo	56
15.2	Objetivos de aprendizagem	56
15.3	Conceitos teóricos	57
15.3.1	Isolamento de contas AWS por ambiente	57
15.3.2	Esteira por produto	57
15.3.3	Fluxo de promoção entre ambientes	57
15.4	Implementação orientada por passos	57
15.4.1	Passo 1 — Estrutura de repositório por produto	57
15.4.2	Passo 2 — Configurar <i>backend</i> Terraform por ambiente	58
15.4.3	Passo 3 — Workflow de CI (pull request)	59
15.4.4	Passo 4 — Workflow de deploy com promoção controlada	60
15.5	Plano de testes do capítulo	61
15.5.1	Teste funcional	61

15.5.2	Teste de desempenho	61
15.5.3	Teste de qualidade	61
15.6	Critérios de aceite	62
15.7	Espaço para expansão iterativa	62
16	Qualidade, Segurança e Governança nas Esteiras de Produto	63
16.1	Escopo do capítulo	63
16.2	Objetivos de aprendizagem	63
16.3	Conceitos teóricos	64
16.3.1	Quality gates nas esteiras bancárias	64
16.3.2	Segurança estática: Python e Terraform	64
16.3.3	Princípio do menor privilégio em IAM	64
16.4	Implementação orientada por passos	65
16.4.1	Passo 1 — Configurar quality gate de testes e lint	65
16.4.2	Passo 2 — Testes de integração contra ambiente <code>dev</code>	65
16.4.3	Passo 3 — Análise de segurança estática	67
16.4.4	Passo 4 — Role IAM com menor privilégio via Terraform	67
16.5	Plano de testes do capítulo	69
16.5.1	Teste funcional	69
16.5.2	Teste de conformidade	69
16.6	Critérios de aceite	70
16.7	Espaço para expansão iterativa	70
17	Frontend Web com CloudFront para Operação da API e Ingestão RAG	71
17.1	Escopo do capítulo	71
17.2	Objetivos de aprendizagem	71
17.3	Conceitos teóricos	72
17.3.1	Frontend estático como superfície de operação	72
17.3.2	CloudFront + S3 privado	72
17.3.3	Chat assíncrono e ingestão RAG em uma única UI	72
17.4	Implementação orientada por passos	73
17.4.1	Passo 1 — Estruturar os assets do frontend	73
17.4.2	Passo 2 — Publicar a UI em CloudFront	73
17.4.3	Passo 3 — Implementar o fluxo de consulta assíncrona	74
17.4.4	Passo 4 — Implementar envio de documentos para RAG	74
17.4.5	Passo 5 — Planejar evolução para PDF e DOCX	74
17.5	Plano de testes do capítulo	75
17.5.1	Teste funcional	75
17.5.2	Teste de validação em produção (AWS)	75
17.5.3	Teste de segurança	76

17.6 Critérios de aceite	76
17.7 Espaço para expansão iterativa	76
A Template de Capítulo	77
A.1 Objetivos do capítulo	77
A.2 Implementação prática	77
A.3 Plano de testes	77
A.4 Critérios de aceite	77
B Glossário Matemático e Técnico	78

Capítulo 1

Fundamentos de IA e Modelos de Linguagem de Grande Escala

1.1 Escopo do capítulo

Este capítulo apresenta os conceitos fundamentais que sustentam toda a arquitetura desenvolvida ao longo do livro: modelos de linguagem de grande escala (do inglês *Large Language Models* — LLMs), mecanismos de atenção, tokenização, janelas de contexto e engenharia de *prompts*. Ao dominar esses conceitos, o leitor adquire a base teórica necessária para compreender as decisões de projeto adotadas nos capítulos subsequentes, em especial a escolha do modelo *Claude Sonnet* via *Amazon Bedrock* como núcleo de inferência da plataforma.

1.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Descrever a arquitetura *Transformer* e o papel do mecanismo de atenção na geração de texto;
- Explicar o processo de tokenização e calcular o consumo estimado de *tokens* de uma requisição;
- Formular *prompts* eficazes utilizando as técnicas *zero-shot*, *few-shot* e *chain-of-thought* (CoT)¹;
- Identificar os parâmetros de amostragem — temperatura, *top-p* e *top-k* — e seu impacto na qualidade e na diversidade das respostas;

¹*Chain-of-Thought* é uma técnica de *prompting* que instrui o modelo a exibir o raciocínio passo a passo antes de produzir a resposta final (1).

- Invocar o modelo *Claude Sonnet* via API *Amazon Bedrock* a partir de código Python.

1.3 Conceitos teóricos

1.3.1 Arquitetura Transformer

A arquitetura *Transformer*, proposta por [2], revolucionou o processamento de linguagem natural ao substituir redes recorrentes pelo mecanismo de *self-attention*, que permite ao modelo relacionar quaisquer dois tokens da sequência de entrada independentemente de sua distância posicional. Formalmente, a atenção é calculada como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1.1)$$

em que Q , K e V são as matrizes de consulta (*query*), chave (*key*) e valor (*value*), respectivamente, e d_k é a dimensão da chave, utilizada como fator de normalização para estabilizar os gradientes durante o treinamento. A função *softmax*² normaliza os produtos escalados, produzindo pesos de atenção que somam 1. O empilhamento de múltiplas camadas de atenção multi-cabeça (*multi-head attention*) confere ao modelo a capacidade de capturar padrões sintáticos, semânticos e pragmáticos simultaneamente.

LLMs são modelos *Transformer* de único-encoder-decoder ou somente-decoder treinados em corpora massivos mediante o objetivo de previsão do próximo *token* (*next-token prediction*). O volume de dados e parâmetros — tipicamente da ordem de bilhões — confere a esses modelos capacidades emergentes não observadas em escalas menores, como raciocínio lógico, tradução sem supervisão explícita e geração de código (3).

1.3.2 Tokenização

Tokens são as unidades mínimas de processamento de um LLM, obtidos por meio de algoritmos de segmentação subpalavra, como *Byte-Pair Encoding* (BPE)³ e *SentencePiece*⁴. A relação entre palavras e *tokens* varia conforme o vocabulário do modelo; como regra prática, estima-se que 1.000 palavras em português equivalem a aproximadamente 1.300–1.500 *tokens* no modelo *Claude Sonnet*.

O custo operacional e a latência de inferência são diretamente proporcionais ao número de *tokens* processados, tornando a contagem prévia um fator crítico no dimensionamento da plataforma. No projeto descrito neste livro, adota-se a biblioteca `tiktoken` (4) como estimador de *tokens* nos benchmarks de desempenho apresentados no Capítulo 11.

² $\sigma(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ — converte o vetor de *logits* em distribuição de probabilidade. Definição formal no Apêndice B, p. 80.

³Definição formal e pseudocódigo no Apêndice B, p. 78.

⁴Definição e comparação com BPE no Apêndice B, p. 79.

1.3.3 Janela de Contexto

A janela de contexto (*context window*) define o número máximo de *tokens* — somando-se entrada e saída — que o modelo processa em uma única requisição. O *Claude Sonnet 4.5*, utilizado na plataforma, suporta até 200.000 *tokens* de entrada, o que possibilita a ingestão de documentos extensos sem necessidade de segmentação agressiva. Todavia, o custo por *token* de entrada é inferior ao de saída; portanto, o projeto limita a janela de contexto injetada pelo pipeline RAG — descrito no Capítulo 5 — a 8.000 *tokens*, valor determinado empiricamente como suficiente para manter alta qualidade de resposta sem comprometer a latência.

1.3.4 Parâmetros de Amostragem

A distribuição de probabilidade sobre o vocabulário na etapa de decodificação é modulada por três parâmetros principais:

Temperatura (τ) Reescala os logits⁵ antes do *softmax*⁶: valores baixos ($\tau \rightarrow 0$) tornam a distribuição mais pontuda, favorecendo respostas deterministas; valores altos ($\tau > 1$) aumentam a entropia e, portanto, a diversidade da geração.

Top- p (amostragem por núcleo) Restringe o vocabulário ativo ao menor conjunto de *tokens* cuja probabilidade acumulada supera p ; valores típicos situam-se entre 0,85 e 0,95.

Top- k Limita a amostragem aos k *tokens* mais prováveis; é frequentemente combinado com *top- p* para eliminar candidatos de baixa probabilidade.

No projeto, adota-se $\tau = 0,3$ nas consultas RAG de modo **segmento** e **persona**, priorizando consistência e fidelidade ao contexto recuperado. Consultas de modo **twín** utilizam $\tau = 0,5$ para permitir maior variabilidade na reprodução do comportamento individual.

1.4 Implementação orientada por passos

1.4.1 Passo 1 — Configuração das credenciais AWS

O acesso ao *Amazon Bedrock* requer credenciais IAM com a permissão `bedrock:InvokeModel`. Recomenda-se utilizar perfis nomeados via *AWS CLI* para evitar a gestão manual de chaves no código:

⁵*Logits*: valores reais não normalizados produzidos pela última camada linear do modelo. Definição formal no Apêndice B, p. 79.

⁶Função $\sigma(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ que converte logits em distribuição de probabilidade. Definição completa no Apêndice B, p. 80.

```

1 aws configure --profile langchain-dev
2 # AWS Access Key ID: <ID>
3 # AWS Secret Access Key: <CHAVE>
4 # Default region name: us-east-1
5 # Default output format: json

```

Listing 1.1: Configuração de perfil AWS CLI.

1.4.2 Passo 2 — Primeira chamada ao modelo

O cliente Bedrock é instanciado via `boto3`, e a requisição é formatada conforme a API *Messages* da Anthropic:

```

1 import boto3, json
2
3 MODEL_ID = "us.anthropic.claude-sonnet-4-5-20250514-v1:0"
4 REGION   = "us-east-1"
5
6 client = boto3.client("bedrock-runtime", region_name=REGION)
7
8 payload = {
9     "anthropic_version": "bedrock-2023-05-31",
10    "max_tokens": 512,
11    "temperature": 0.3,
12    "messages": [
13        {
14            "role": "user",
15            "content": "Explique o conceito de tokenizacao em
16                        LLMs "
17                        "em ate tres paragrafos, estilo
18                        academico."
19        }
20    ]
21 }
22
23 response = client.invoke_model(
24     modelId=MODEL_ID,
25     body=json.dumps(payload),
26     contentType="application/json",
27     accept="application/json"
28 )

```

```

28 body    = json.loads(response["body"].read())
29 texto   = body["content"][0]["text"]
30 tokens  = body["usage"]                # input_tokens, output_tokens
31
32 print(texto)
33 print(f"\nTokens consumidos: entrada={tokens['input_tokens']},
34       "
35       f"saida={tokens['output_tokens']}")

```

Listing 1.2: Primeira chamada ao *Claude Sonnet* via Bedrock.

Verifica-se que o campo `usage` retornado pela API contém a contagem exata de *tokens* de entrada e saída, permitindo o rastreamento preciso dos custos de inferência.

1.4.3 Passo 3 — Engenharia de *prompts*

A qualidade da resposta é sensível à estrutura do *prompt*. O padrão adotado no projeto separa o *system prompt* — que define o papel, o tom e as restrições do modelo — das mensagens de usuário, conforme recomendado pela documentação da Anthropic (5):

```

1  payload = {
2      "anthropic_version": "bedrock-2023-05-31",
3      "max_tokens": 1024,
4      "temperature": 0.3,
5      "system": (
6          "Voce e um assistente especializado em analise de
7            clientes "
8          "bancarios. Responda em portugues do Brasil, estilo
9            formal, "
10         "com base exclusivamente no contexto fornecido. "
11         "Nao invente informacoes ausentes do contexto."
12     ),
13     "messages": [
14         {
15             "role": "user",
16             "content": (
17                 "<context>\n{contexto_recuperado}\n</context>\n"
18                 "\n"
19                 "Pergunta: {pergunta_usuario}"
20             )
21         }
22     ]
23 }

```

Listing 1.3: Estrutura de *system prompt* e mensagem de usuário.

O uso de delimitadores XML (`<context>`) é recomendado para que o modelo distinga claramente o contexto recuperado da instrução, reduzindo o risco de alucinações.

1.4.4 Passo 4 — Técnica *few-shot*

Exemplos de entrada-saída inseridos no *prompt* (*few-shot examples*) calibram o formato e o registro das respostas sem necessidade de ajuste fino (*fine-tuning*). No projeto, cada modo de consulta — **segmento**, **persona** e **twin** — utiliza de dois a três exemplos curados que demonstram a estrutura esperada da resposta, conforme detalhado no Capítulo 8.

1.5 Plano de testes do capítulo

1.5.1 Teste funcional

- **TC-01:** Invocar o modelo com *prompt* mínimo e verificar que o campo `content[0].text` está preenchido e não é vazio.
- **TC-02:** Passar um *prompt* com temperatura $\tau = 0$ e verificar que duas execuções consecutivas produzem saídas idênticas (determinismo).
- **TC-03:** Exceder o limite de *max_tokens* e confirmar que a resposta é truncada com o motivo `max_tokens` no campo `stop_reason`.

1.5.2 Teste de desempenho

- **TP-01:** Medir a latência de 10 chamadas consecutivas com *prompt* de 500 *tokens*; critério: mediana $\leq 3,0$ s (região `us-east-1`).
- **TP-02:** Calcular o custo estimado por 1.000 requisições de 1.000 *tokens* de entrada e 500 de saída com base na tabela de preços *on-demand* do Bedrock.

1.5.3 Teste de qualidade de resposta

- **TQ-01:** Formular cinco perguntas de referência sobre tokenização e avaliar as respostas quanto à precisão factual (escala 1–5).
- **TQ-02:** Verificar que a resposta a uma pergunta fora do contexto retorna a expressão de ausência de informação, sem alucinação.

1.6 Critérios de aceite

- TC-01, TC-02 e TC-03 executam sem erros bloqueantes.
- TP-01: mediana de latência $\leq 3,0$ s confirmada em execução real (evidenciar com *log* de timestamps).
- TQ-01: pontuação média $\geq 4,0/5,0$.
- TQ-02: zero alucinações detectadas nas cinco perguntas fora de contexto.

1.7 Espaço para expansão iterativa

- **v1.1** — Adicionar comparação de modelos: *Claude Haiku* (baixo custo) vs. *Claude Sonnet* (equilíbrio) vs. *Claude Opus* (máxima qualidade), com métricas de latência e custo por token.
- **v1.2** — Incluir diagrama da arquitetura *Transformer* (codificador e decodificador) adaptado ao estilo do livro.
- **Lacuna** — Avaliar impacto de diferentes estratégias de truncamento de contexto (*sliding window* vs. sumarização hierárquica) na qualidade das respostas.

Capítulo 2

Ambiente, Ferramentas e Reprodutibilidade

2.1 Escopo do capítulo

Este capítulo descreve a configuração completa do ambiente de desenvolvimento adotado no projeto, abrangendo gerenciamento de dependências Python, credenciais AWS, provisionamento de infraestrutura via *Terraform* e organização do repositório. O objetivo é garantir que qualquer colaborador consiga reproduzir os experimentos descritos ao longo do livro a partir de um ambiente zerado, sem ambiguidades de versão ou de configuração.

2.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Configurar um ambiente Python 3.11 isolado com todas as dependências do projeto fixadas em versão exata;
- Organizar credenciais AWS de forma segura utilizando perfis nomeados e variáveis de ambiente;
- Provisionar a infraestrutura AWS (Lambda, DynamoDB, SQS, OpenSearch, Neptune) via *Terraform* em uma única execução reproduzível;
- Executar os *notebooks* Jupyter do projeto e verificar os resultados esperados de cada seção.

2.3 Conceitos e prerequisites

2.3.1 Versões e dependências

A reprodutibilidade de experimentos com LLMs é comprometida por três fontes principais de não-determinismo: (i) versão do modelo de inferência, (ii) versão das bibliotecas de orquestração e (iii) configuração do ambiente de execução. Para mitigar esses riscos, o projeto adota as seguintes práticas:

- Todas as dependências são declaradas com versão exata no arquivo `requirements.txt` (não utilizar `~=` ou `>=` sem limites superiores);
- O modelo de inferência é referenciado pelo identificador versionado `us.anthropic.claude-sonnet-4-5` e não por alias genérico como `claude-sonnet-latest`;
- A versão do Python (3.11.x) e do *Terraform* (1.7.x) são documentadas no arquivo `README.md` na raiz do repositório.

2.3.2 Estrutura do repositório

O repositório é organizado segundo o princípio de separação de responsabilidades, distinguindo código de aplicação, infraestrutura como código e artefatos de documentação:

```
1 langchain/  
2   infraestrutura/           # Terraform (modulos por servico AWS  
3   )  
4   modules/  
5     lambda/src/           # Codigo das funcoes Lambda  
6     opensearch/  
7     neptune/  
8     dynamodb/  
9     sqs/  
10    api_gateway/  
11  livro/                   # Fonte LaTeX do livro  
12    chapters/  
13  decisoes_projeto/       # ADRs (Architecture Decision  
14    Records)  
15  tests/                   # Testes unitarios e de integracao  
16  langchain.ipynb          # Notebook didatico principal  
    demo_api.ipynb          # Notebook de demonstracao da API  
    README.md
```

Listing 2.1: Estrutura de diretórios do repositório.

2.4 Implementação orientada por passos

2.4.1 Passo 1 — Clonar o repositório e criar o ambiente virtual

```
1 git clone https://github.com/<org>/langchain-rag.git
2 cd langchain-rag
3
4 # Criar e ativar o ambiente virtual
5 python3.11 -m venv .venv
6 source .venv/bin/activate          # Linux/macOS
7 # .venv\Scripts\Activate.ps1      # Windows PowerShell
8
9 # Instalar dependências fixadas
10 pip install --upgrade pip
11 pip install -r requirements.txt
```

Listing 2.2: Criação do ambiente virtual Python isolado.

As dependências mínimas para executar os notebooks do projeto são listadas abaixo com as versões verificadas:

```
1 boto3==1.34.69
2 langchain==0.2.16
3 langchain-community==0.2.16
4 langgraph==0.2.4
5 faiss-cpu==1.8.0
6 tiktoken==0.7.0
7 jupyter==1.0.0
8 nbformat==5.10.4
9 pypdf==4.3.1
10 requests==2.32.3
```

Listing 2.3: Dependências principais (`requirements.txt`).

2.4.2 Passo 2 — Configurar credenciais AWS

O acesso aos serviços AWS é controlado por funções IAM (*Identity and Access Management*). Para desenvolvimento local, utiliza-se um perfil nomeado com permissões mínimas necessárias:

```
1 aws configure --profile langchain-dev
2 # AWS Access Key ID:      <ID gerado no IAM>
3 # AWS Secret Access Key: <CHAVE>
4 # Default region name:    sa-east-1
```

```
5 # Default output format: json
6
7 # Exportar o perfil como variavel de ambiente no shell
8 export AWS_PROFILE=langchain-dev
```

Listing 2.4: Configuração do perfil AWS para desenvolvimento.

Importante: credenciais não devem ser *hardcoded* no código nem versionadas no repositório. Recomenda-se incluir `.env` e `*.tfvars` no `.gitignore`.

2.4.3 Passo 3 — Provisionar a infraestrutura com Terraform

A infraestrutura AWS é descrita como código no diretório `infraestrutura/`. O provisionamento completo — incluindo VPC, API Gateway, filas SQS, funções Lambda, tabela DynamoDB, domínio OpenSearch e cluster Neptune — é executado com:

```
1 cd infraestrutura/
2 terraform init
3 terraform validate
4 terraform plan -out=deploy.tfplan
5 terraform apply deploy.tfplan
```

Listing 2.5: Provisionamento da infraestrutura via Terraform.

Após a conclusão, os *outputs* relevantes — URL da API, *endpoints* do OpenSearch e do Neptune, nome da tabela DynamoDB — são exibidos no terminal e podem ser capturados com:

```
1 terraform output -json > ../model_data.json
```

Listing 2.6: Extração de outputs do Terraform.

2.4.4 Passo 4 — Executar os notebooks

Com o ambiente configurado e a infraestrutura provisionada, os *notebooks* são iniciados via Jupyter ou diretamente no VS Code com a extensão *Jupyter*:

```
1 jupyter lab --no-browser --port=8888
```

Listing 2.7: Inicialização do servidor Jupyter.

Recomenda-se executar os notebooks na ordem natural dos capítulos — `langchain.ipynb` (conceitos) e `demo_api.ipynb` (demonstração da API) — pois as células iniciais de cada *notebook* definem constantes e funções utilizadas nas células subsequentes.

2.5 Plano de testes do capítulo

2.5.1 Teste funcional

- **TC-01:** Executar `pip install -r requirements.txt` em ambiente zerado e verificar ausência de conflitos de versão (exit code 0).
- **TC-02:** Executar `python -c "import boto3, langchain, langgraph, faiss"` e verificar que nenhum `ImportError` é lançado.
- **TC-03:** Executar `aws sts get-caller-identity` com o perfil configurado e verificar que o ARN retornado pertence à conta correta.
- **TC-04:** Executar `terraform validate` no diretório de infraestrutura e verificar mensagem `Success! The configuration is valid.`

2.5.2 Teste de desempenho

- **TP-01:** Medir o tempo total de `terraform apply` em ambiente limpo; critério: conclusão em menos de 20 minutos para o conjunto completo de módulos.
- **TP-02:** Medir o tempo de inicialização do kernel Jupyter até a execução da primeira célula de importação; critério: menos de 30 s.

2.5.3 Teste de qualidade

- **TQ-01:** Verificar que o arquivo `.gitignore` exclui `*.tfstate`, `*.tfvars` e `.env` do versionamento.
- **TQ-02:** Confirmar que nenhuma chave de acesso AWS está presente no histórico de commits via `git log --all --full-diff | grep -i "AKIA"`.

2.6 Critérios de aceite

- TC-01 a TC-04 concluídos sem erros em um ambiente Windows 11 com Python 3.11 e Terraform 1.7.
- TP-01: `terraform apply` concluído em menos de 20 minutos (evidenciar com *log* de execução).
- TQ-01 e TQ-02 aprovados; nenhuma credencial versionada detectada.

2.7 Espaço para expansão iterativa

- **v2.1** — Adicionar *devcontainer* Docker para eliminar dependência de instalação local de Python e Terraform.
- **v2.2** — Incluir pipeline CI/CD (GitHub Actions) que execute `terraform validate` e os testes unitários a cada *pull request*.
- **Lacuna** — Avaliar o uso de *Terraform Workspaces* para isolar ambientes de desenvolvimento, homologação e produção sem duplicar código de infraestrutura.

Capítulo 3

Agentes com LangGraph e Memória

3.1 Escopo do capítulo

Este capítulo apresenta a construção de agentes de IA orientados a grafos de estado utilizando a biblioteca *LangGraph* (6), que estende o *LangChain* (7) com suporte nativo a ciclos, persistência de estado e orquestração de múltiplos atores. O ciclo ReAct (*Reason + Act*), proposto por 8, é adotado como padrão de raciocínio; o agente alterna entre etapas de análise (*thought*), invocação de ferramentas (*action*) e observação dos resultados (*observation*) até satisfazer o critério de parada. A gestão de memória — tanto de curto prazo (histórico de mensagens) quanto de médio prazo (vetor semântico FAISS) — é descrita e implementada ao longo do capítulo.

3.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Modelar o comportamento de um agente como um grafo de estados tipado com `TypedDict` e arestas condicionais;
- Implementar o ciclo ReAct com o *LangGraph* conectando um nó de raciocínio ao *Claude Sonnet* e nós de ação a ferramentas externas;
- Gerenciar a memória de curto prazo por meio do histórico de mensagens (`MessagesState`) e persistir o estado entre invocações com `MemorySaver`;
- Integrar memória vetorial FAISS como fonte de recuperação semântica para subsidiar as decisões do agente.

3.3 Conceitos teóricos

3.3.1 Grafo de estados e nós de processamento

No *LangGraph*, um agente é modelado como um grafo dirigido em que cada nó é uma função Python que recebe e retorna um dicionário de estado. O estado é definido por uma classe `TypedDict` que declara explicitamente os campos trocados entre os nós, garantindo segurança de tipos em tempo de compilação:

```
1 from typing import TypedDict, Annotated, Sequence
2 from langchain_core.messages import BaseMessage
3 from langgraph.graph.message import add_messages
4
5 class EstadoAgente(TypedDict):
6     messages: Annotated[Sequence[BaseMessage], add_messages]
7     contexto_rag: str
8     tentativas: int
```

Listing 3.1: Definição do estado do agente.

O redutor `add_messages` garante que novas mensagens sejam acumuladas em vez de substituírem o histórico, preservando o contexto de conversação entre turnos. O campo `messages` corresponde ao tipo `MessagesState` — abstração de conveniência do *LangGraph* que combina a lista de mensagens com o redutor `add_messages`.

3.3.2 Ciclo ReAct

O ciclo ReAct é implementado como uma sequência de arestas no grafo: o nó `raciocinar` invoca o LLM e gera uma mensagem que pode conter chamadas de ferramenta (*tool calls*); o nó `executar_ferramentas` processa essas chamadas e retorna as observações; a aresta condicional `deve_continuar` verifica se o LLM solicitou mais ferramentas ou produziu uma resposta final, determinando o próximo nó. Conforme 8, a interlocução entre raciocínio e ação aumenta significativamente a taxa de sucesso em tarefas de question-answering multietapas em comparação com abordagens puramente encadeadas.

3.3.3 Persistência de estado com MemorySaver

O `MemorySaver` é um *checkpoint* que persiste o estado do grafo entre invocações em memória local (adequado para desenvolvimento) ou em banco de dados externo (adequado para produção). Cada sessão de conversação é identificada por um `thread_id`, permitindo que o agente retome o contexto após interrupções sem reprocessar o histórico completo.

3.4 Implementação orientada por passos

3.4.1 Passo 1 — Definir as ferramentas do agente

Ferramentas são funções Python decoradas com `@tool`, que expõem ao LLM uma interface descritiva com *docstring* e assinatura de tipos:

```
1 from langchain_core.tools import tool
2
3 @tool
4 def buscar_perfil_cliente(cliente_id: str) -> dict:
5     """Recupera o perfil de um cliente bancario pelo ID.
6     Retorna segmento, score de credito e produtos ativos."""
7     # Implementacao: consulta DynamoDB / OpenSearch
8     ...
9
10 @tool
11 def calcular_score_credito(renda: float, dividas: float) ->
    float:
12     """Calcula o score de credito simplificado dado renda e
13     dividas.
14     Retorna um valor entre 0.0 e 1.0."""
15     return max(0.0, min(1.0, (renda - dividas) / renda))
16 ferramentas = [buscar_perfil_cliente, calcular_score_credito]
```

Listing 3.2: Definição de ferramentas com `@tool`.

3.4.2 Passo 2 — Construir o grafo ReAct

O grafo é instanciado com `StateGraph` — classe central do *LangGraph* que recebe o tipo de estado e gerencia nós e arestas. O modelo de linguagem é acessado via `ChatBedrock`, cliente *LangChain* para o *Amazon Bedrock*. A execução das ferramentas é delegada ao `ToolNode`, nó pré-construído que processa automaticamente as *tool calls* retornadas pelo LLM:

```
1 import boto3
2 from langchain_aws import ChatBedrock
3 from langchain_core.messages import SystemMessage
4 from langgraph.graph import StateGraph, END
5 from langgraph.prebuilt import ToolNode
6 from langgraph.checkpoint.memory import MemorySaver
7
```

```

8 MODEL_ID = "us.anthropic.claude-sonnet-4-5-20250514-v1:0"
9
10 llm = ChatBedrock(
11     model_id=MODEL_ID,
12     region_name="us-east-1",
13     model_kwargs={"temperature": 0.3, "max_tokens": 2048}
14 ).bind_tools(ferramentas)
15
16 SYSTEM = SystemMessage(content=(
17     "Voce e um assistente financeiro especializado. "
18     "Use as ferramentas disponiveis para responder com precisao
19     . "
20     "Nunca invente dados ausentes nas ferramentas."
21 ))
22
23 def raciocinar(state: EstadoAgente) -> dict:
24     resposta = llm.invoke([SYSTEM] + list(state["messages"]))
25     return {"messages": [resposta]}
26
27 def deve_continuar(state: EstadoAgente) -> str:
28     ultima = state["messages"][-1]
29     return "ferramentas" if ultima.tool_calls else END
30
31 grafo = StateGraph(EstadoAgente)
32 grafo.add_node("raciocinar", raciocinar)
33 grafo.add_node("ferramentas", ToolNode(ferramentas))
34 grafo.set_entry_point("raciocinar")
35 grafo.add_conditional_edges("raciocinar", deve_continuar)
36 grafo.add_edge("ferramentas", "raciocinar")
37
38 agente = grafo.compile(checkpointer=MemorySaver())

```

Listing 3.3: Construção do grafo de agente ReAct com *LangGraph*.

3.4.3 Passo 3 — Invocar o agente com persistência de contexto

```

1 from langchain_core.messages import HumanMessage
2
3 config = {"configurable": {"thread_id": "sessao-cliente-001"}}
4
5 # Primeiro turno
6 resultado = agente.invoke(

```

```

7     {"messages": [HumanMessage(content="Qual o perfil do
8         cliente C-0042?")]}},
9     config=config
10 )
11 print(resultado["messages"][-1].content)
12
13 # Segundo turno -- agente retoma o historico do thread
14 resultado = agente.invoke(
15     {"messages": [HumanMessage(content="E o score de credito
16         dele?")]}},
17     config=config
18 )
19 print(resultado["messages"][-1].content)

```

Listing 3.4: Invocação do agente com `thread_id` para persistência de contexto.

Observa-se que no segundo turno o agente não necessita receber novamente o contexto do cliente; o `MemorySaver` recupera o estado persistido do `thread_id` e o *Claude Sonnet* pode referenciar a resposta anterior sem ambiguidade.

3.4.4 Passo 4 — Integrar memória vetorial FAISS

Para consultas que exigem recuperação semântica de documentos históricos — políticas, contratos, notas de entrevista — integra-se um índice FAISS¹ como ferramenta do agente:

```

1 from langchain_community.vectorstores import FAISS
2 from langchain_aws import BedrockEmbeddings
3
4 embeddings = BedrockEmbeddings(
5     model_id="amazon.titan-embed-text-v2:0",
6     region_name="us-east-1"
7 )
8 indice = FAISS.load_local("faiss_index", embeddings,
9                             allow_dangerous_deserialization=True
10 )
11
12 @tool
13 def buscar_documentos(consulta: str) -> str:
14     """Busca documentos relevantes no repositório corporativo.
15     Use quando precisar de políticas, procedimentos ou
16     contratos."""

```

¹Facebook AI Similarity Search (FAISS) é uma biblioteca de busca por similaridade de vetores de alta dimensão (9).

```
15 docs = indice.similarity_search(consulta, k=3)
16 return "\n---\n".join(d.page_content for d in docs)
17
18 ferramentas.append(buscar_documentos)
```

Listing 3.5: Recuperação semântica via FAISS integrada como ferramenta.

3.5 Plano de testes do capítulo

3.5.1 Teste funcional

- **TC-01:** Invocar o agente com uma pergunta que requer exatamente uma chamada de ferramenta e verificar que o grafo percorre o ciclo `raciocinar` → `ferramentas` → `raciocinar` → `END`.
- **TC-02:** Verificar que o histórico é preservado entre dois turnos consecutivos com o mesmo `thread_id`.
- **TC-03:** Perguntar sobre informação não disponibilizada em nenhuma ferramenta e verificar que o agente responde com declaração de ausência, sem alucinação.
- **TC-04:** Trocar o `thread_id` e verificar que o contexto não vaza entre sessões distintas.

3.5.2 Teste de desempenho

- **TP-01:** Medir a latência de 10 invocações com uma chamada de ferramenta cada; critério: mediana $\leq 4,0$ s.
- **TP-02:** Medir o número de *tokens* acumulados após 5 turnos consecutivos e verificar que não ultrapassa 20.000 *tokens* de entrada (janela de contexto controlada).

3.5.3 Teste de qualidade de resposta

- **TQ-01:** Avaliar 10 pares pergunta-resposta com o LLM-as-Judge (detalhado no Capítulo 12); critério: pontuação média de fidelidade $\geq 8,0/10$.
- **TQ-02:** Verificar que o agente não recorre a ferramentas desnecessariamente em perguntas que podem ser respondidas com o histórico da conversa.

3.6 Critérios de aceite

- TC-01 a TC-04 aprovados sem erros bloqueantes.

- TP-01: mediana de latência $\leq 4,0$ s confirmada (evidenciar com *timestamps* no notebook).
- TQ-01: pontuação média $\geq 8,0/10$ em 10 perguntas de referência.

3.7 Espaço para expansão iterativa

- **v3.1** — Substituir o `MemorySaver` em memória pelo `AsyncPostgresSaver` — *checkpoint* assíncrono baseado em PostgreSQL — para persistência em produção com múltiplos usuários concorrentes.
- **v3.2** — Implementar supervisão hierárquica com um agente orquestrador (*supervisor*) que delega subproblemas a agentes especializados (financeiro, jurídico, atendimento), conforme o padrão *multi-agent supervisor* do *LangGraph*.
- **Lacuna** — Investigar o impacto da ordem das ferramentas no *prompt* sobre a taxa de chamadas incorretas (*hallucinated tool calls*).

Capítulo 4

Tools, Skills e MCP

4.1 Escopo do capítulo

Ferramentas customizadas, protocolos MCP e integracao com fontes externas.

4.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

4.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

4.4 Plano de testes do capítulo

4.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

4.4.2 Teste de desempenho

- Medir latencia ponta a ponta.
- Medir uso estimado de tokens por execucao.

4.4.3 Teste de qualidade de resposta

- Medir aderencia ao contexto recuperado.
- Medir consistencia da resposta com o objetivo do usuario.

4.5 Criterios de aceite

- Implementacao executa sem erros bloqueantes.
- Testes definidos no capitulo atingem resultado esperado.
- Evidencias de execucao sao anexadas pelos autores.

4.6 Espaco para expansao iterativa

- Notas de versao do capitulo.
- Melhorias planejadas.
- Lacunas para investigacao futura.

Capítulo 5

Fundamentos de RAG

5.1 Escopo do capítulo

Ingestao de conhecimento, chunking, embeddings e retrieval inicial.

5.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

5.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

5.4 Plano de testes do capítulo

5.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

5.4.2 Teste de desempenho

- Medir latencia ponta a ponta.
- Medir uso estimado de tokens por execucao.

5.4.3 Teste de qualidade de resposta

- Medir aderencia ao contexto recuperado.
- Medir consistencia da resposta com o objetivo do usuario.

5.5 Criterios de aceite

- Implementacao executa sem erros bloqueantes.
- Testes definidos no capitulo atingem resultado esperado.
- Evidencias de execucao sao anexadas pelos autores.

5.6 Espaco para expansao iterativa

- Notas de versao do capitulo.
- Melhorias planejadas.
- Lacunas para investigacao futura.

Capítulo 6

RAG Corporativo e Arquitetura de Dados

6.1 Escopo do capítulo

Este capítulo apresenta, em formato de aula-laboratório, a implementação de um pipeline *Retrieval-Augmented Generation* (RAG) para contexto bancário, com foco em fontes heterogêneas, indexação, recuperação e governança de dados. O objetivo prático consiste em construir um fluxo no qual documentos corporativos e entrevistas de clientes são convertidos em contexto recuperável para agentes, reduzindo alucinação e aumentando fidelidade da resposta.

6.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Diferenciar conhecimento parametricamente aprendido pelo LLM e conhecimento recuperado por RAG;
- Modelar documentos de negócio para indexação em OpenSearch;
- Implementar ingestão textual para os tipos `documento` e `entrevista`;
- Validar, com casos de teste, se a resposta do agente utiliza o contexto recuperado de forma verificável.

6.3 Conceitos teoricos

6.3.1 RAG em ambiente regulado

Em ambiente bancario, perguntas de negocio dependem de politicas, regras de produto, condicoes de canal e historico de relacionamento. Essas informacoes mudam com frequencia e nao devem depender exclusivamente do conhecimento interno do modelo. O padrao RAG injeta contexto em tempo de execucao, desacoplando atualizacao documental de novo *fine-tuning*.

6.3.2 Tipos de documento no projeto

O projeto adota dois tipos principais:

- **documento:** politicas, FAQ, regras de oferta e materiais normativos;
- **entrevista:** texto de voz do cliente ou de grupo, utilizado para os modos *persona* e *twin*.

Essa separacao permite governanca por origem e facilita auditoria de qual fonte sustentou determinada resposta.

6.4 Implementacao orientada por passos

6.4.1 Passo 1 — Preparar colecao de documentos

Nesta etapa, organiza-se um conjunto minimo para reproducao da aula:

```
1 dados_rag/  
2   documentos/  
3     politica_investimentos_2026.txt  
4     faq_cartao_credito.txt  
5   entrevistas/  
6     cliente_C00042.txt  
7     cluster_1_focus_group.txt
```

Listing 6.1: Colecao minima para laboratorio de RAG corporativo.

Cada arquivo deve conter texto contínuo, sem metadados sensíveis. Campos de identificacao (cliente ou cluster) devem ser enviados explicitamente no payload.

6.4.2 Passo 2 — Ingerir texto no endpoint /documentos

O endpoint de ingestao esta definido no contrato OpenAPI e aceita JSON. No laboratorio, utiliza-se a chamada abaixo:

```
1 curl -X POST "$API_URL/documentos" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "tipo": "documento",  
5     "titulo": "Politica de Investimentos 2026",  
6     "texto": "Clientes conservadores priorizam liquidez diaria  
7       ..."  
8   }'
```

Listing 6.2: Exemplo de ingestao de documento de politica no RAG.

Para entrevista individual:

```
1 curl -X POST "$API_URL/documentos" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "tipo": "entrevista",  
5     "titulo": "Entrevista cliente C00042",  
6     "cliente_id": "C00042",  
7     "texto": "Prefiro resolver tudo no app e evitar taxas altas  
8       ..."  
9   }'
```

Listing 6.3: Ingestao de entrevista associada a cliente.

6.4.3 Passo 3 — Verificar chunking e indexacao

A funcao de ingestao fragmenta textos em *chunks* e os grava em indices OpenSearch distintos para cada finalidade. Conceitualmente, o fluxo corresponde a:

```
1 if tipo == "entrevista":  
2     chunks = chunkar(texto)  
3     indexar("entrevistas-clientes", chunks, metadata)  
4 else:  
5     chunks = chunkar(texto)  
6     indexar("documentos-knowledge", chunks, metadata)
```

Listing 6.4: Fluxo simplificado da indexacao no ingester.

O resultado esperado da API e um JSON contendo `chunks_indexados > 0`.

6.4.4 Passo 4 — Consumir o contexto em consulta real

Apos indexar, executa-se uma pergunta de negocio no endpoint assíncrono:

```
1 curl -X POST "$API_URL/query" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "modo": "segmento",  
5     "pergunta": "Qual produto devo oferecer para perfil  
6       conservador?",  
7     "dados_cliente": {  
8       "idade": 55,  
9       "renda_mensal": 18000,  
10      "saldo_medio": 120000,  
11      "transacoes_mes": 25,  
12      "score_credito": 820,  
13      "num_produtos": 7  
14    }  
  },'
```

Listing 6.5: Consulta de negocio para verificar uso do contexto RAG.

Em seguida, consulta-se `/status/{request_id}` ate COMPLETED e compara-se a resposta com os documentos enviados.

6.4.5 Passo 5 — Resultado completo da aula

Ao final da pratica, o ambiente deve conter:

- indice `documentos-knowledge` com politicas e FAQ;
- indice `entrevistas-clientes` com voz de cliente/grupo;
- resposta do agente sustentada por contexto recuperado;
- evidencias de payload, `request_id` e resposta final arquivadas.

6.5 Plano de testes do capitulo

6.5.1 Teste funcional

- **TC-01:** Ingerir um documento e verificar `statusCode = 200` e `chunks_indexados > 0`.
- **TC-02:** Ingerir uma entrevista com `cliente_id` e validar persistencia no indice de entrevistas.

- **TC-03:** Executar consulta `modo=segmento` e confirmar resposta final sem erro de processamento.

6.5.2 Teste de desempenho

- **TP-01:** Medir tempo de ingestao de 10 documentos de 2 KB; criterio: mediana $\leq 2,5$ s por documento.
- **TP-02:** Medir latencia de resposta assíncrona ponta a ponta; criterio: $p95 \leq 20$ s em ambiente dev.

6.5.3 Teste de qualidade de resposta

- **TQ-01:** Verificar, em 10 perguntas, se a resposta cita explicitamente contexto compatível com os documentos indexados.
- **TQ-02:** Avaliar fidelidade com LLM-as-Judge (Capitulo 12); criterio: media $\geq 8,0/10$.

6.6 Critérios de aceite

- TC-01 a TC-03 aprovados sem erro bloqueante.
- TP-01 e TP-02 dentro dos limiares definidos.
- TQ-01 com aderencia documental observavel em pelo menos 80% dos casos.

6.7 Espaco para expansao iterativa

- **v6.1** — Adicionar suporte a PDF/DOCX com pipeline de extracao.
- **v6.2** — Incluir versionamento de documento e expiracao por vigencia.
- **Lacuna** — Investigar *reranking* para aumentar precisao top-k.

Capítulo 7

Pipeline Assíncrono em AWS

7.1 Escopo do capítulo

Este capítulo descreve, como laboratório completo, a arquitetura assíncrona usada para superar o limite de timeout do API Gateway e processar consultas RAG com LLM em tempo potencialmente superior a 29 segundos. O fluxo adotado é: API Gateway → Lambda Controller → SQS → Lambda Worker → DynamoDB → Lambda Status.

7.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Explicar por que o processamento síncrono falha para pipelines longos;
- Implementar o padrão de enfileiramento com SQS e persistência de status em DynamoDB;
- Construir endpoints `/query` e `/status/{request_id}` com contrato consistente;
- Validar escalabilidade horizontal do worker e observabilidade de estados.

7.3 Conceitos teóricos

7.3.1 Padrão fire-and-poll

A estratégia adotada é *fire-and-poll*: o cliente envia a requisição, recebe um `request_id` imediatamente e consulta o estado periodicamente até a conclusão. Essa abordagem desacopla o tempo de resposta da API do tempo de computação do pipeline.

7.3.2 Estados de processamento

O registro no DynamoDB utiliza estados típicos:

- PENDING: requisição recebida e enfileirada;
- PROCESSING: worker em execução;
- COMPLETED: resposta pronta;
- FAILED: erro irreversível com mensagem para auditoria.

7.4 Implementação orientada por passos

7.4.1 Passo 1 — Submeter consulta e enfileirar

O endpoint POST /query recebe payload e delega o trabalho ao SQS, retornando 202 Accepted com identificador único:

```
1 request_id = str(uuid.uuid4())
2 put_item_dynamodb(request_id, status="PENDING", payload=body)
3 enviar_mensagem_sqs(request_id=request_id, payload=body)
4
5 return {
6     "statusCode": 202,
7     "body": json.dumps({"request_id": request_id, "status": "
8         PENDING"})
9 }
```

Listing 7.1: Fluxo simplificado do controller: enfileirar e responder rápido.

7.4.2 Passo 2 — Processar no worker

O worker consome mensagens da fila, executa o pipeline e grava o resultado:

```
1 for msg in mensagens_sqs:
2     request_id = msg["request_id"]
3     atualizar_status(request_id, "PROCESSING")
4     try:
5         resposta = executar_pipeline_rag(msg["payload"])
6         salvar_resultado(request_id, status="COMPLETED", result
7             =resposta)
8     except Exception as exc:
9         salvar_resultado(request_id, status="FAILED", error=str
10             (exc))
```

Listing 7.2: Fluxo simplificado do worker com update de status.

7.4.3 Passo 3 — Consultar status

O endpoint GET /status/{request_id} consulta DynamoDB e retorna o estado corrente para a interface web ou cliente programatico.

```
1 curl "$API_URL/status/$REQUEST_ID"
```

Listing 7.3: Consulta de status no laboratorio.

7.4.4 Passo 4 — Teste ponta a ponta de laboratorio

Sequencia recomendada da aula:

1. Enviar POST /query com payload valido;
2. Armazenar request_id;
3. Executar polling a cada 2 s;
4. Registrar transicao de estados ate COMPLETED.

```
1 REQUEST_ID="..."
2 while true; do
3     RESP=$(curl -s "$API_URL/status/$REQUEST_ID")
4     echo "$RESP"
5     echo "$RESP" | findstr COMPLETED >nul && break
6     echo "$RESP" | findstr FAILED >nul && break
7     timeout /t 2 >nul
8 done
```

Listing 7.4: Script minimo de polling com shell.

7.4.5 Passo 5 — Resultado completo da aula

Ao final, o aluno deve observar:

- aceitacao imediata da requisicao no controller;
- processamento desacoplado no worker;
- resultado final consultavel por request_id;
- ausencia de timeout no cliente para cargas demoradas.

7.4.6 Passo 6 — Evidencia operacional via frontend CloudFront

Em validacao de producao assistida por frontend web (CloudFront), observou-se o comportamento esperado do padrao *fire-and-poll*. Para duas requisicoes reais (modos **persona** e **twin**), o fluxo evoluiu de **PENDING** para **PROCESSING** e finalizou em **COMPLETED**, sem expiracao no cliente.

- **Persona:** `request_id: <ddbc5210-63fc-47bc-a274-283e60be84bc>`.

Estado final **COMPLETED**;

- **Twin:** `request_id: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>`.

Estado final **COMPLETED**.

Essa evidencia reforca que a arquitetura assincrona permanece estavel quando acessada por uma interface de navegador, preservando o mesmo contrato de API utilizado por scripts de laboratorio.

7.5 Plano de testes do capitulo

7.5.1 Teste funcional

- **TC-01:** `POST /query` retorna 202 com `request_id` valido.
- **TC-02:** status evolui de **PENDING** para **PROCESSING** e depois **COMPLETED**.
- **TC-03:** para payload invalido, status final deve ser **FAILED** com mensagem de erro rastreavel.

7.5.2 Teste de desempenho

- **TP-01:** tempo de resposta do controller ≤ 1 s para 100 requisicoes seriadas.
- **TP-02:** throughput do worker com lote de 500 mensagens sem perda de consistencia em DynamoDB.

7.5.3 Teste de qualidade de resposta

- **TQ-01:** comparar 20 respostas assincronas com baseline sincrono local e verificar equivalencia semantica.
- **TQ-02:** validar que o modo de polling nao altera o conteudo final da resposta.

7.6 Critérios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites acordados para **dev**.
- TQ-01 com equivalencia media $\geq 95\%$ frente ao baseline.
- Validacao real de polling no frontend com status final **COMPLETED** em modos distintos de consulta.

7.7 Espaco para expansao iterativa

- **v7.1** — Trocar polling por WebSocket para notificacao ativa.
- **v7.2** — Adicionar *dead-letter queue* e retentativas.
- **Lacuna** — Modelar SLA distinto por tipo de consulta e modo de agente.

Capítulo 8

Segmentacao, Personas e Digital Twins

8.1 Escopo do capitulo

Este capitulo implementa, em formato de aula pratica guiada, a transformacao de dados de clientes em tres tipos de agentes de pesquisa: agente por segmento, agente *persona* e *digital twin*. O fluxo combina clusterizacao com K-Means, geracao de contexto RAG por grupo e criacao sob demanda de agentes para respostas em primeira ou terceira pessoa conforme o objetivo analitico.

8.2 Objetivos de aprendizagem

Ao concluir este capitulo, o leitor sera capaz de:

- Treinar e aplicar K-Means para segmentacao de clientes;
- Construir documentos RAG por cluster e por cliente;
- Diferenciar os modos `segmento`, `persona` e `twin`;
- Validar, com casos controlados, a consistencia de resposta de cada modo.

8.3 Conceitos teoricos

8.3.1 Tres superficies de analise

O projeto adota tres superficies complementares:

- **Segmento:** resposta gerencial em terceira pessoa com base no perfil medio do cluster;

- **Persona:** arquétipo nomeado de um cluster, respondendo em primeira pessoa representativa;
- **Twin:** simulacao individual, baseada em dados reais do cliente e contexto RAG filtrado por `cliente_id`.

8.3.2 Features de clusterizacao

O vetor de entrada utiliza, minimamente, as variaveis: `idade`, `renda_mensal`, `saldo_medio`, `transacoes_mes`, `score_credito` e `num_produtos`. Antes do K-Means, aplica-se padronizacao com *StandardScaler*.

8.4 Implementacao orientada por passos

8.4.1 Passo 1 — Gerar base sintetica e executar clustering

O laboratorio inicia com dados sinteticos para reproducao local:

```
1 df = gerar_dados_sinteticos(n=1000)
2 kmeans, scaler, perfis = executar_clustering(df, n_clusters=4)
3
4 cliente_exemplo = {
5     "idade": 55,
6     "renda_mensal": 18000,
7     "saldo_medio": 120000,
8     "transacoes_mes": 25,
9     "score_credito": 820,
10    "num_produtos": 7,
11 }
12
13 cluster_id = int(kmeans.predict(scaler.transform([[
14     cliente_exemplo["idade"],
15     cliente_exemplo["renda_mensal"],
16     cliente_exemplo["saldo_medio"],
17     cliente_exemplo["transacoes_mes"],
18     cliente_exemplo["score_credito"],
19     cliente_exemplo["num_produtos"],
20 ]]))[0])
```

Listing 8.1: Treino de clusterizacao e previsao de cluster.

8.4.2 Passo 2 — Criar contexto RAG por cluster

Cada cluster gera documentos de contexto com perfil medio, produtos e estrategia de abordagem. O material e indexado no OpenSearch para recuperacao posterior.

```
1 for cid, perfil in perfis.iterrows():
2     docs = _gerar_docs_cluster(cid, perfil)
3     indexar_docs_cluster(cid, docs) # abstrai o backend (FAISS
                                     /OpenSearch)
```

Listing 8.2: Geracao de documentos RAG por cluster.

8.4.3 Passo 3 — Instanciar agente persona

O modo *persona* reaproveita contexto do cluster, mas muda voz e comportamento por meio de *system prompt* dedicado.

```
1 agente_persona = _criar_agente_persona(
2     cluster_id=cluster_id,
3     perfil=perfis.loc[cluster_id],
4     retriever=retriever_cluster(cluster_id),
5     model=modelo_llm,
6 )
7
8 resposta = agente_persona.invoke({"pergunta": "O que voce
                                     espera de um cartao?"})
```

Listing 8.3: Criacao de agente persona por cluster.

8.4.4 Passo 4 — Criar digital twin sob demanda

No modo *twin*, o agente nao depende da rotulagem de cluster; ele e construido com base no registro individual e no retriever filtrado por *cliente_id*.

```
1 twin = criar_twin_sob_demanda(
2     cliente_id="C00042",
3     dados_cliente=cliente_exemplo,
4     model=modelo_llm,
5 )
6
7 out = twin.invoke({"pergunta": "Como posso melhorar meu score
                                     de credito?"})
```

Listing 8.4: Twin sob demanda com contexto individual.

8.4.5 Passo 5 — Chamar a API em cada modo

O laboratorio fecha o ciclo testando os tres modos pela API:

```
1 # segmento
2 curl -X POST "$API_URL/query" -H "Content-Type: application/
   json" -d '{
3   "modo": "segmento", "pergunta": "Qual oferta para este
   cliente?", "dados_cliente": {...}
4 }'
5
6 # persona
7 curl -X POST "$API_URL/query" -H "Content-Type: application/
   json" -d '{
8   "modo": "persona", "cluster_id": 1, "pergunta": "O que voce
   valoriza em investimentos?"
9 }'
10
11 # twin
12 curl -X POST "$API_URL/query" -H "Content-Type: application/
   json" -d '{
13   "modo": "twin", "cliente_id": "C00042", "pergunta": "Qual o
   melhor proximo produto para mim?", "dados_cliente": {...}
14 }'
```

Listing 8.5: Chamadas de validacao dos modos segmento, persona e twin.

8.4.6 Passo 6 — Resultado completo da aula

Ao final, o aluno deve conseguir demonstrar:

- classificacao automatica do cliente em cluster;
- resposta gerencial (segmento), arquetipica (persona) e individual (twin);
- coerencia entre contexto recuperado e estilo de resposta por modo.

8.4.7 Passo 7 — Evidencias de execucao em producao

Em teste de smoke test na AWS, executado pelo frontend CloudFront integrado a API Gateway, os modos **persona** e **twin** apresentaram conclusao com estado **COMPLETED**, mantendo aderencia de voz em primeira pessoa e recomendacoes compativeis com o contexto recuperado.

- **Persona** (`cluster_id=1`): `request_id`: <ddbc5210-63fc-47bc-a274-283e60be84bc>. Estado final COMPLETED;
- **Twin** (`cliente_id=C00042`): `request_id`: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>. Estado final COMPLETED.

Como resultado, observa-se que a estrategia de segmentacao e a construcao do agente individual permanecem operacionalmente consistentes no ambiente implantado, e nao apenas em execucao local de laboratorio.

8.5 Plano de testes do capitulo

8.5.1 Teste funcional

- **TC-01**: classificar 50 clientes e verificar ausencia de erro de dimensao no *scaler*/K-Means.
- **TC-02**: executar `modo=persona` com `cluster_id` explicito e retornar resposta em primeira pessoa.
- **TC-03**: executar `modo=twin` com `cliente_id` e confirmar uso de contexto individual.

8.5.2 Teste de desempenho

- **TP-01**: tempo de classificacao por cliente ≤ 50 ms.
- **TP-02**: latencia mediana da inferencia por modo ≤ 5 s em dev.

8.5.3 Teste de qualidade de resposta

- **TQ-01**: em 10 perguntas por modo, verificar aderencia de tom (3a pessoa em segmento; 1a pessoa em persona/twin).
- **TQ-02**: avaliar fidelidade com LLM-as-Judge; criterio: media $\geq 8,0/10$ por modo.

8.6 Criterios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites definidos.
- TQ-01 com conformidade de voz em pelo menos 90% dos casos.
- Execucao real em producao com modos `persona` e `twin` finalizando em COMPLETED.

8.7 Espaço para expansao iterativa

- **v8.1** — Incorporar cache LRU para twins de alta frequencia.
- **v8.2** — Evoluir para clusterizacao periodica com janela movel.
- **Lacuna** — Investigar validacao causal para evitar vies de segmentacao.

Capítulo 9

GraphRAG com Neptune

9.1 Escopo do capítulo

Este capítulo apresenta, em formato de laboratório, a extensão do RAG para contexto em grafo, combinando recuperação textual com inferência de relações multi-hop em Amazon Neptune. O objetivo consiste em responder perguntas que dependem de conexões entre entidades (cliente, produto, segmento, persona, canal), não apenas de similaridade textual.

9.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Explicar quando RAG textual é insuficiente para consultas relacionais;
- Popular um grafo com entidades e arestas de negócio;
- Consultar contexto relacional com OpenCypher;
- Combinar contexto de grafo e contexto textual na resposta final do agente.

9.3 Conceitos teóricos

9.3.1 Do RAG textual ao GraphRAG

No RAG tradicional, a recuperação prioriza trechos semanticamente próximos. Entretanto, perguntas como “quais produtos conectam cliente de alto risco a segmento de renda média com canal digital?” exigem travessia de arestas. O GraphRAG introduz camada relacional para atender essas consultas sem perder o contexto documental.

9.3.2 Fontes de contexto combinadas

No pipeline de producao, o worker pode combinar:

- **RAG BM25 textual** em OpenSearch;
- **Snapshot de grafo replicado** em indice sincronizado;
- **Consulta Neptune live** para relacoes recentes.

Esse desenho reduz latencia media, mas preserva capacidade de consulta profunda quando necessario.

9.4 Implementacao orientada por passos

9.4.1 Passo 1 — Popular o grafo de conhecimento

Inicialmente, insere-se um conjunto minimo de entidades e relacoes:

```
1 python popular_neptune.py
2 # alternativa em ambiente Lambda/VPC:
3 # python seed_neptune_lambda.py
```

Listing 9.1: Carga inicial do grafo para o laboratorio.

Resultado esperado: vertices de clientes, segmentos, personas e produtos, com arestas representando afinidade, posse e recomendacao.

9.4.2 Passo 2 — Ingerir documentos para mapeamento textual

A camada textual continua necessaria para justificativas e detalhes descritivos. Portanto, indexa-se material de apoio no OpenSearch:

```
1 python indexar_documento.py \
2   --tipo documento \
3   --titulo "Politica de Cross-sell 2026" \
4   --arquivo dados_rag/documentos/politica_investimentos_2026.
   txt
```

Listing 9.2: Ingestao de documento no pipeline GraphRAG.

9.4.3 Passo 3 — Executar consulta relacional no Neptune

Consulta de exemplo em OpenCypher para recuperar produtos associados ao segmento de um cliente:

```
1 MATCH (c:Cliente {cliente_id: "C00042"})-[:PERTENCE_A]->(s:
    Segmento)
2 MATCH (s)-[:RECOMENDA]->(p:Produto)
3 RETURN c.cliente_id, s.nome AS segmento, collect(p.nome) AS
    produtos
```

Listing 9.3: Consulta OpenCypher de laboratorio.

9.4.4 Passo 4 — Fundir contexto textual e relacional

No worker, a estratégia de fusão pode ser representada como:

```
1 ctx_texto = recuperar_opensearch(pergunta, top_k=4)
2 ctx_grafo = consultar_neptune(pergunta, cliente_id)
3
4 contexto_final = {
5     "texto": ctx_texto,
6     "grafo": ctx_grafo,
7 }
8
9 resposta = llm.invoke(prompt_com_contexto(contexto_final,
    pergunta))
```

Listing 9.4: Composição simplificada de contexto híbrido.

9.4.5 Passo 5 — Validar em consulta de negócio

Após a fusão, executa-se pergunta multi-hop que dependa de relações e documentos:

```
1 curl -X POST "$API_URL/query" \
2   -H "Content-Type: application/json" \
3   -d '{
4     "modo": "twin",
5     "cliente_id": "C00042",
6     "pergunta": "Quais produtos se conectam ao meu segmento e
7     como justificam essa oferta?",
8     "dados_cliente": {
9       "idade": 35,
10      "renda_mensal": 3200,
11      "saldo_medio": 800,
12      "transacoes_mes": 12,
13      "score_credito": 520,
```

```
13     "num_produtos": 2
14   }
15 },
```

Listing 9.5: Consulta final do laboratorio GraphRAG.

9.4.6 Passo 6 — Resultado completo da aula

Ao final, o aluno deve comprovar:

- grafo populado com entidades e relacoes auditaveis;
- contexto textual e relacional utilizados de forma complementar;
- resposta final com coerencia de recomendacao e justificativa.

9.5 Plano de testes do capitulo

9.5.1 Teste funcional

- **TC-01:** executar carga inicial do grafo e validar contagem minima de vertices e arestas.
- **TC-02:** rodar consulta OpenCypher de referencia e verificar retorno nao vazio para cliente conhecido.
- **TC-03:** executar pergunta multi-hop e confirmar resposta com elementos relacionais e textuais.

9.5.2 Teste de desempenho

- **TP-01:** p95 de consulta OpenCypher ≤ 400 ms em dev.
- **TP-02:** latencia ponta a ponta com contexto hibrido ≤ 8 s em mediana.

9.5.3 Teste de qualidade de resposta

- **TQ-01:** para 10 perguntas multi-hop, verificar que a resposta menciona relacoes corretas entre entidades.
- **TQ-02:** avaliar fidelidade com LLM-as-Judge; criterio: media $\geq 8,5/10$.

9.6 Critérios de aceite

- TC-01 a TC-03 aprovados.
- TP-01 e TP-02 dentro dos limites estabelecidos.
- TQ-01 com consistência relacional em pelo menos 80% dos casos.

9.7 Espaço para expansão iterativa

- **v9.1** — Adicionar estratégia de *path ranking* para priorizar trilhas de maior relevância.
- **v9.2** — Incluir explicações citáveis por aresta para auditoria.
- **Lacuna** — Avaliar impacto de replicação Neptune→OpenSearch na frescura de dados em cenários de alta mutação.

Capítulo 10

Confiabilidade, Observabilidade e Operacao

10.1 Escopo do capitulo

Indice de confiabilidade, logs, traces, monitoracao e resiliencia.

10.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

10.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

10.4 Plano de testes do capitulo

10.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

10.4.2 Teste de desempenho

- Medir latência ponta a ponta.
- Medir uso estimado de tokens por execução.

10.4.3 Teste de qualidade de resposta

- Medir aderência ao contexto recuperado.
- Medir consistência da resposta com o objetivo do usuário.

10.5 Critérios de aceite

- Implementação executa sem erros bloqueantes.
- Testes definidos no capítulo atingem resultado esperado.
- Evidências de execução são anexadas pelos autores.

10.6 Espaço para expansão iterativa

- Notas de versão do capítulo.
- Melhorias planejadas.
- Lacunas para investigação futura.

Capítulo 11

Metricas de Avaliacao de Mercado

11.1 Escopo do capitulo

Metricas de qualidade para RAG e LLMs, incluindo relevancia e fidelidade.

11.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

11.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

11.4 Plano de testes do capitulo

11.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

11.4.2 Teste de desempenho

- Medir latencia ponta a ponta.
- Medir uso estimado de tokens por execucao.

11.4.3 Teste de qualidade de resposta

- Medir aderencia ao contexto recuperado.
- Medir consistencia da resposta com o objetivo do usuario.

11.5 Criterios de aceite

- Implementacao executa sem erros bloqueantes.
- Testes definidos no capitulo atingem resultado esperado.
- Evidencias de execucao sao anexadas pelos autores.

11.6 Espaco para expansao iterativa

- Notas de versao do capitulo.
- Melhorias planejadas.
- Lacunas para investigacao futura.

Capítulo 12

LLM as Judge

12.1 Escopo do capítulo

Avaliação automatizada com juiz LLM, rubricas e calibração de notas.

12.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementação concreta do projeto.
- Definir evidências objetivas de qualidade e funcionamento.

12.3 Implementação orientada por passos

1. Preparação do ambiente e insumos.
2. Implementação guiada do fluxo principal.
3. Validação dos resultados esperados.
4. Registro de decisões técnicas e trade-offs.

12.4 Plano de testes do capítulo

12.4.1 Teste funcional

- Definir casos de entrada e saída esperada.
- Verificar comportamento nominal e casos de erro.

12.4.2 Teste de desempenho

- Medir latencia ponta a ponta.
- Medir uso estimado de tokens por execucao.

12.4.3 Teste de qualidade de resposta

- Medir aderencia ao contexto recuperado.
- Medir consistencia da resposta com o objetivo do usuario.

12.5 Criterios de aceite

- Implementacao executa sem erros bloqueantes.
- Testes definidos no capitulo atingem resultado esperado.
- Evidencias de execucao sao anexadas pelos autores.

12.6 Espaco para expansao iterativa

- Notas de versao do capitulo.
- Melhorias planejadas.
- Lacunas para investigacao futura.

Capítulo 13

Golden Dataset e Regressao de Qualidade

13.1 Escopo do capitulo

Construcao de dataset dourado, suites de regressao e gates de release.

13.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

13.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

13.4 Plano de testes do capitulo

13.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

13.4.2 Teste de desempenho

- Medir latência ponta a ponta.
- Medir uso estimado de tokens por execução.

13.4.3 Teste de qualidade de resposta

- Medir aderência ao contexto recuperado.
- Medir consistência da resposta com o objetivo do usuário.

13.5 Critérios de aceite

- Implementação executa sem erros bloqueantes.
- Testes definidos no capítulo atingem resultado esperado.
- Evidências de execução são anexadas pelos autores.

13.6 Espaço para expansão iterativa

- Notas de versão do capítulo.
- Melhorias planejadas.
- Lacunas para investigação futura.

Capítulo 14

Roadmap de Producao e Evolucao Continua

14.1 Escopo do capitulo

Plano de maturidade tecnica, operacao e melhoria continua.

14.2 Objetivos de aprendizagem

- Entender os conceitos nucleares do tema.
- Aplicar o tema em uma implementacao concreta do projeto.
- Definir evidencias objetivas de qualidade e funcionamento.

14.3 Implementacao orientada por passos

1. Preparacao do ambiente e insumos.
2. Implementacao guiada do fluxo principal.
3. Validacao dos resultados esperados.
4. Registro de decisoes tecnicas e trade-offs.

14.4 Plano de testes do capitulo

14.4.1 Teste funcional

- Definir casos de entrada e saida esperada.
- Verificar comportamento nominal e casos de erro.

14.4.2 Teste de desempenho

- Medir latência ponta a ponta.
- Medir uso estimado de tokens por execução.

14.4.3 Teste de qualidade de resposta

- Medir aderência ao contexto recuperado.
- Medir consistência da resposta com o objetivo do usuário.

14.5 Critérios de aceite

- Implementação executa sem erros bloqueantes.
- Testes definidos no capítulo atingem resultado esperado.
- Evidências de execução são anexadas pelos autores.

14.6 Espaço para expansão iterativa

- Notas de versão do capítulo.
- Melhorias planejadas.
- Lacunas para investigação futura.

Capítulo 15

Esteiras de CI/CD Multi-Ambiente para Produtos AWS Bancários

15.1 Escopo do capítulo

Este capítulo descreve a organização e a operação das esteiras de integração contínua e entrega contínua (*Continuous Integration/Continuous Delivery* — CI/CD) adotadas nos produtos AWS do projeto bancário. A estratégia baseia-se em três ambientes isolados — `dev`, `homol` e `prod` — cada qual mapeado para uma conta AWS independente, garantindo isolamento de recursos, permissões e dados entre estágios do ciclo de vida do software. A infraestrutura de cada produto é integralmente descrita em módulos *Terraform* versionados, e os fluxos de automação são implementados como *Git Workflows* (*GitHub Actions* ou *GitLab CI*).

15.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Compreender a arquitetura de três ambientes AWS separados e as implicações de isolamento de conta para sistemas bancários;
- Estruturar esteiras de CI/CD independentes por produto AWS, com estágios de build, validação, homologação e produção claramente definidos;
- Configurar *workspaces* Terraform mapeados a ambientes e *state backends* segregados por conta;
- Implementar controles de promoção entre ambientes com aprovação humana obrigatória antes de `homol` e `prod`.

15.3 Conceitos teóricos

15.3.1 Isolamento de contas AWS por ambiente

O modelo de três contas AWS separadas — uma por ambiente — constitui prática recomendada pelo *AWS Well-Architected Framework* para sistemas financeiros regulados. O isolamento por conta oferece:

- **Blast radius controlado:** falhas, vazamentos de credenciais ou configurações equivocadas em `dev` não afetam `prod`;
- **Políticas IAM independentes:** cada conta define seus próprios limites de permissão sem interferência cruzada;
- **Segregação de dados:** dados reais de clientes presentes apenas em `prod`; `dev` e `homol` operam exclusivamente com dados sintéticos ou anonimizados.

O fluxo de promoção entre ambientes é descrito na subseção seguinte.

15.3.2 Esteira por produto

Cada produto AWS — Lambda, API Gateway, DynamoDB, SQS, OpenSearch, Neptune — mantém seu próprio repositório Git com conjunto de arquivos de *workflow* independente. Essa separação previne acoplamento entre produtos e permite que equipes distintas evoluam cadências de deploy sem bloquear umas às outras.

15.3.3 Fluxo de promoção entre ambientes

O ciclo de promoção segue o padrão:

1. **Feature branch** → **dev**: merge disparado por *pull request* aprovado; pipeline executa lint, testes unitários e `terraform plan`.
2. **dev** → **homol**: merge na branch `main`; exige aprovação do responsável de QA e pipeline verde.
3. **homol** → **prod**: *tag* de release semântica (`vMAJOR.MINOR.PATCH`); exige aprovação de dois revisores autorizados e janela de manutenção registrada.

15.4 Implementação orientada por passos

15.4.1 Passo 1 — Estrutura de repositório por produto

A organização recomendada para o repositório de um produto AWS é:

```

1 produto-lambda-segmentacao/
2   .github/
3     workflows/
4       ci.yml           # lint, testes, plan -- toda PR
5       deploy-dev.yml  # deploy automatico em dev
6       deploy-homol.yml # deploy com aprovacao QA
7       deploy-prod.yml # deploy com aprovacao dupla + tag
8   infraestrutura/
9     main.tf
10    variables.tf
11    outputs.tf
12    backend.tf         # configuracao S3 + DynamoDB lock
13    envs/
14      dev.tfvars
15      homol.tfvars
16      prod.tfvars      # sem segredos -- apenas referencias
17  src/
18    lambda_handler.py
19    tests/
20      test_unit.py
21      test_integration.py
22  requirements.txt
23  requirements-dev.txt

```

Listing 15.1: Estrutura de repositório de produto AWS bancário.

15.4.2 Passo 2 — Configurar *backend* Terraform por ambiente

Cada ambiente utiliza um *bucket* S3 e uma tabela DynamoDB dedicados para armazenar e bloquear o *state*:

```

1 # backend.tf
2 terraform {
3   backend "s3" {
4     bucket      = "tfstate-${var.ambiente}-123456789"
5     key         = "produto-lambda-segmentacao/terraform.
6                 tfstate"
7     region      = "us-east-1"
8     dynamodb_table = "tfstate-lock-${var.ambiente}"
9     encrypt     = true
10  }
11 }

```

Listing 15.2: Backend Terraform com lock por conta AWS.

O parâmetro `encrypt = true` garante criptografia do *state file* em repouso, exigência obrigatória para repositórios bancários que possam conter ARNs, IDs de recursos ou referências a segredos.

15.4.3 Passo 3 — Workflow de CI (pull request)

O workflow de CI é disparado em toda *pull request* aberta contra qualquer branch protegida:

```
1 # .github/workflows/ci.yml
2 name: CI
3
4 on:
5   pull_request:
6     branches: [main, develop]
7
8 jobs:
9   quality:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v4
13
14       - name: Setup Python
15         uses: actions/setup-python@v5
16         with:
17           python-version: "3.11"
18
19       - name: Instalar dependencias
20         run: pip install -r requirements-dev.txt
21
22       - name: Lint (flake8 + black)
23         run: |
24           flake8 src/ --max-line-length=100
25           black --check src/
26
27       - name: Testes unitarios
28         run: pytest src/tests/test_unit.py --cov=src --cov-fail
29             -under=70
30
31       - name: Seguranca estatica (bandit)
32         run: bandit -r src/ -ll
```

```

32
33     - name: Terraform validate
34       uses: hashicorp/setup-terraform@v3
35       with:
36         terraform_version: "1.7.0"
37     - run: |
38         cd infraestrutura
39         terraform init -backend=false
40         terraform fmt -check -recursive
41         terraform validate

```

Listing 15.3: GitHub Actions: workflow de CI para produto AWS.

15.4.4 Passo 4 — Workflow de deploy com promoção controlada

O deploy em `homol` e `prod` utiliza o mecanismo de *environment protection rules* do GitHub Actions, que bloqueia a execução até que revisores autorizados aprovem:

```

1  # .github/workflows/deploy-homol.yml
2  name: Deploy Homol
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    deploy:
10     runs-on: ubuntu-latest
11     environment: homologacao # protection rule: 1 aprovador
12                        QA
13     permissions:
14       id-token: write # OIDC para assume role sem
15                        chave estatica
16       contents: read
17
18     steps:
19       - uses: actions/checkout@v4
20
21       - name: Configurar credenciais AWS via OIDC
22         uses: aws-actions/configure-aws-credentials@v4
23         with:
24           role-to-assume: arn:aws:iam::HOMOL_ACCOUNT_ID:role/

```

```

23         GitHubActionsRole
24         aws-region: us-east-1
25     - name: Terraform apply (homol)
26       run: |
27         cd infraestrutura
28         terraform init
29         terraform workspace select homol
30         terraform apply -var-file=envs/homol.tfvars -auto-approve

```

Listing 15.4: GitHub Actions: deploy em homol com aprovacao.

A autenticação via *OIDC* (*OpenID Connect*) elimina a necessidade de chaves de acesso estáticas (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`) armazenadas como *secrets* do repositório, reduzindo significativamente a superfície de ataque.

15.5 Plano de testes do capítulo

15.5.1 Teste funcional

- **TC-01:** Abrir uma PR de feature, introduzir um erro de lint intencional e verificar que o job `quality` falha antes do deploy.
- **TC-02:** Fazer merge na branch `main` sem aprovação do QA e verificar que o job `deploy-homol` fica bloqueado aguardando aprovação.
- **TC-03:** Criar uma tag `v1.0.0` sem os dois revisores autorizados e verificar que o deploy em prod não prossegue.

15.5.2 Teste de desempenho

- **TP-01:** Medir o tempo total da pipeline de CI (checkout a validação Terraform); critério: ≤ 8 min.
- **TP-02:** Medir o tempo de `terraform apply` em `homol` para o produto Lambda de segmentação; critério: ≤ 5 min.

15.5.3 Teste de qualidade

- **TQ-01:** Executar `checkov` contra os módulos Terraform e verificar zero falhas de nível CRITICAL.

- **TQ-02:** Validar que o *state file* de **prod** está criptografado e o acesso ao bucket S3 é restrito apenas à role de CI/CD.

15.6 Critérios de aceite

- TC-01 a TC-03 aprovados sem intervenção manual.
- TP-01: pipeline de CI ≤ 8 min evidenciada com *timestamps* do GitHub Actions.
- TQ-01: zero falhas CRITICAL no **checkov**.
- Nenhuma chave de acesso estática AWS presente nos *secrets* do repositório; autenticação exclusivamente via OIDC.

15.7 Espaço para expansão iterativa

- **v15.1** — Implementar *drift detection*: job agendado que executa **terraform plan** diariamente em **prod** e alerta via SNS caso detecte desvio entre o *state* e a infraestrutura real.
- **v15.2** — Adotar *GitOps* com *ArgoCD* ou *Flux* para sincronização declarativa de manifests Kubernetes, se a plataforma evoluir para EKS.
- **Lacuna** — Investigar o impacto de *feature flags* no ciclo de promoção, permitindo ativar funcionalidades seletivamente por conta AWS sem novo deploy.

Capítulo 16

Qualidade, Segurança e Governança nas Esteiras de Produto

16.1 Escopo do capítulo

Este capítulo detalha os requisitos de qualidade mínima, os padrões de segurança e as práticas de governança obrigatórios em todas as esteiras de produto da plataforma bancária. Enquanto o Capítulo 15 aborda a estrutura de ambientes e o fluxo de promoção, este capítulo concentra-se nos *quality gates* — pontos de controle que um artefato deve transpor antes de avançar ao próximo estágio — e nos controles de conformidade exigidos pela regulação bancária brasileira, em especial a Resolução CMN nº 4.893/2021 do Banco Central do Brasil.

16.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Definir e configurar *quality gates* para testes unitários, de integração e de segurança estática em pipelines AWS;
- Aplicar análise estática de segurança com **bandit** (Python) e **checkov** / **tfsec** (Terraform);
- Implementar o princípio do menor privilégio em roles IAM criadas via Terraform;
- Configurar auditoria centralizada com CloudTrail e alertas de desvio de conformidade.

16.3 Conceitos teóricos

16.3.1 Quality gates nas esteiras bancárias

Um *quality gate* é uma condição binária — aprovado/reprovado — que bloqueia o avanço do pipeline caso um limiar mínimo não seja atingido. A Tabela 16.1 apresenta os gates obrigatórios, os limiares adotados e as ferramentas responsáveis por cada verificação.

Tabela 16.1: Quality gates obrigatórios por estágio de pipeline bancário.

Estágio	Gate	Limiar	Ferramenta
PR / CI	Cobertura de testes unitários	$\geq 70\%$	<code>pytest -cov</code>
PR / CI	Lint e formatação	zero erros	<code>flake8</code> , <code>black</code>
PR / CI	Segurança estática Python	zero HIGH	<code>bandit</code>
PR / CI	Segurança Terraform	zero CRITICAL	<code>checkov</code> , <code>tfsec</code>
Homol	Testes de integração	100% pass	<code>pytest</code>
Homol	Drift de infraestrutura	zero desvios	<code>terraform plan</code>
Prod	Aprovação dupla	2 revisores	GitHub Environments
Prod	Janela de manutenção	registrada	Change Management

16.3.2 Segurança estática: Python e Terraform

A análise estática de segurança é executada antes de qualquer deploy, detectando vulnerabilidades conhecidas no código-fonte e nas configurações de infraestrutura:

- **bandit**: verifica o código Python em busca de padrões do OWASP Top 10, como uso de `eval()`, `pickle`, senão constantes em hashes criptográficos e injection vulnerabilities.
- **checkov**: analisa recursos Terraform contra um banco de mais de 1.000 políticas de segurança (CIS Benchmarks, AWS Foundational Security Best Practices).
- **tfsec**: complementa o **checkov** com verificações específicas de misconfigurações AWS, como buckets S3 públicos, security groups com 0.0.0.0/0 e criptografia ausente.

16.3.3 Princípio do menor privilégio em IAM

Roles IAM para funções Lambda e pipelines de CI/CD devem seguir o padrão:

1. Nenhum wildcard (*) em `Action` ou `Resource`.
2. `Condition` com `aws:SourceAccount` restringindo o acesso à conta AWS do ambiente.
3. Política de negação explícita para ações destrutivas (`DeleteTable`, `DeleteBucket`) em ambientes `homol` e `prod`.
4. Revisão semestral automática via AWS Access Analyzer.

16.4 Implementação orientada por passos

16.4.1 Passo 1 — Configurar quality gate de testes e lint

```
1 # Trecho do job "quality" em .github/workflows/ci.yml
2 - name: Testes unitarios com cobertura
3   run: |
4     pytest src/tests/test_unit.py \
5       --cov=src \
6       --cov-report=xml \
7       --cov-fail-under=70 \
8       -v
9
10 - name: Upload cobertura (Codecov)
11   uses: codecov/codecov-action@v4
12   with:
13     files: coverage.xml
14     fail_ci_if_error: true
15
16 - name: Lint (flake8)
17   run: flake8 src/ --max-line-length=100 --statistics
18
19 - name: Formatacao (black)
20   run: black --check --diff src/
```

Listing 16.1: Job de quality gate no GitHub Actions com cobertura e lint.

16.4.2 Passo 2 — Testes de integração contra ambiente dev

Os testes de integração são executados no ambiente `dev` após o deploy bem-sucedido, utilizando dados sintéticos que replicam a estrutura de dados de produção sem expor informações reais de clientes:

```
1 import pytest
2 import boto3
3 import json
4
5 API_URL = "https://api-dev.banco.interno/v1/segmentacao"
6
7 @pytest.fixture(scope="module")
8 def api_token():
9     """Obtem token de teste via Cognito do ambiente dev."""
```

```

10     client = boto3.client("cognito-idp", region_name="us-east-1
11         ")
12     resp = client.initiate_auth(
13         AuthFlow="USER_PASSWORD_AUTH",
14         AuthParameters={
15             "USERNAME": "usuario-teste-ci",
16             "PASSWORD": "Senha-Teste-CI-123!"    # secret do
17             repositório
18         },
19         ClientId="CLIENT_ID_DEV"
20     )
21     return resp["AuthenticationResult"]["IdToken"]
22
23 def test_segmentacao_cliente_valido(api_token):
24     """TC-INT-01: cliente com dados completos deve retornar
25     segmento."""
26     import requests
27     resp = requests.post(
28         API_URL,
29         headers={"Authorization": f"Bearer {api_token}"},
30         json={"cliente_id": "TESTE-001", "produto": "
31             cartao_credito"},
32         timeout=10
33     )
34     assert resp.status_code == 200
35     data = resp.json()
36     assert "segmento" in data
37     assert data["segmento"] in ["PREMIUM", "PADRAO", "BASICO"]
38
39 def test_segmentacao_cliente_invalido(api_token):
40     """TC-INT-02: ID inexistente deve retornar 404."""
41     import requests
42     resp = requests.post(
43         API_URL,
44         headers={"Authorization": f"Bearer {api_token}"},
45         json={"cliente_id": "INEXISTENTE-999"},
46         timeout=10
47     )
48     assert resp.status_code == 404

```

Listing 16.2: Teste de integração para endpoint de segmentação.

16.4.3 Passo 3 — Análise de segurança estática

```

1  # Seguranca Python (bandit)
2  - name: Bandit -- seguranca estatica Python
3  run: |
4      bandit -r src/ \
5          --level HIGH \
6          --confidence HIGH \
7          --format json \
8          --output bandit-report.json
9      # Falha se houver qualquer item HIGH/HIGH
10     python -c "
11         import json, sys
12         r = json.load(open('bandit-report.json'))
13         highs = [i for i in r['results']]
14                 if i['issue_severity'] == 'HIGH'
15                 and i['issue_confidence'] == 'HIGH']
16     if highs:
17         print(f'FALHA: {len(highs)} itens HIGH encontrados')
18         sys.exit(1)
19     print('OK: nenhum item HIGH/HIGH')
20     "
21
22  # Seguranca Terraform (checkov)
23  - name: Checkov -- seguranca Terraform
24  uses: bridgecrewio/checkov-action@master
25  with:
26      directory: infraestrutura/
27      framework: terraform
28      soft_fail: false
29      quiet: true
30      output_format: cli

```

Listing 16.3: Quality gate de segurança: bandit e checkov.

16.4.4 Passo 4 — Role IAM com menor privilégio via Terraform

```

1  # infraestrutura/modules/iam/lambda_segmentacao.tf
2  resource "aws_iam_role" "lambda_segmentacao" {
3      name = "lambda-segmentacao-${var.ambiente}"
4
5      assume_role_policy = jsonencode({

```

```

6     Version = "2012-10-17"
7     Statement = [{
8         Effect      = "Allow"
9         Principal = { Service = "lambda.amazonaws.com" }
10        Action      = "sts:AssumeRole"
11        Condition = {
12            StringEquals = {
13                "aws:SourceAccount" = var.account_id
14            }
15        }
16    }]
17 })
18 }

19
20 resource "aws_iam_role_policy" "lambda_segmentacao_policy" {
21     name = "politica-segmentacao-${var.ambiente}"
22     role = aws_iam_role.lambda_segmentacao.id
23
24     policy = jsonencode({
25         Version = "2012-10-17"
26         Statement = [
27             {
28                 Effect = "Allow"
29                 Action = [
30                     "dynamodb:GetItem",
31                     "dynamodb:Query"
32                 ]
33                 Resource = [
34                     "arn:aws:dynamodb:us-east-1:${var.account_id}:table/
35                         clientes-${var.ambiente}",
36                     "arn:aws:dynamodb:us-east-1:${var.account_id}:table/
37                         clientes-${var.ambiente}/index/*"
38                 ]
39             },
40             {
41                 Effect = "Allow"
42                 Action = [
43                     "secretsmanager:GetSecretValue"
44                 ]
45                 Resource = "arn:aws:secretsmanager:us-east-1:${var.
46                     account_id}:secret:llm/*"
47             }
48         ]
49     })
50 }

```

```

44     },
45     {
46         Effect = "Allow"
47         Action = [
48             "logs:CreateLogGroup",
49             "logs:CreateLogStream",
50             "logs:PutLogEvents"
51         ]
52         Resource = "arn:aws:logs:us-east-1:${var.account_id}:
                    log-group:/aws/lambda/segmentacao-*:*"
53     }
54 ]
55 })
56 }

```

Listing 16.4: Role IAM para Lambda de segmentação com menor privilégio.

Observa-se que nenhuma ação contém wildcard e todos os `Resource` apontam para ARNs específicos do ambiente, parametrizados pelas variáveis `var.account_id` e `var.ambiente`.

16.5 Plano de testes do capítulo

16.5.1 Teste funcional

- **TC-01:** Introduzir um `eval()` no código Python e verificar que o `bandit` reprova o `gate`.
- **TC-02:** Criar um recurso S3 sem criptografia no Terraform e verificar que o `checkov` bloqueia o pipeline.
- **TC-03:** Reduzir a cobertura de testes abaixo de 70% e verificar que o job falha com mensagem explícita.
- **TC-04:** Executar os testes de integração contra `dev` e verificar que TC-INT-01 e TC-INT-02 passam.

16.5.2 Teste de conformidade

- **TConf-01:** Verificar que o CloudTrail está habilitado em todas as três contas AWS (`dev`, `homol`, `prod`) com *trails* multirregionais.
- **TConf-02:** Verificar que o *Access Analyzer* do AWS IAM não reporta acessos externos não intencionais.

- **TConf-03:** Confirmar que nenhuma chave de acesso estática AWS está presente nos *secrets* do repositório via `git-secrets` ou `truffleHog`.

16.6 Critérios de aceite

- TC-01 a TC-04 aprovados.
- TConf-01 a TConf-03 aprovados sem exceção.
- Cobertura de testes $\geq 70\%$ confirmada para todos os produtos em produção.
- Zero findings CRITICAL no `checkov` e HIGH/HIGH no `bandit` nos módulos de produção.

16.7 Espaço para expansão iterativa

- **v16.1** — Integrar *DAST* (*Dynamic Application Security Testing*) com OWASP ZAP no estágio de homologação, executando varredura ativa contra o endpoint do API Gateway de `homol`.
- **v16.2** — Implementar *Software Bill of Materials* (SBOM) automático com `syft` para rastreabilidade de dependências em conformidade com requisitos de auditoria do BACEN.
- **v16.3** — Adotar *policy-as-code* com *Open Policy Agent* (OPA) para centralizar e versionar políticas de segurança como código, reutilizáveis em todos os produtos.
- **Lacuna** — Investigar a adequação ao *CIS AWS Foundations Benchmark* e ao *PCI DSS* v4.0 para produtos que processam dados de cartão.

Capítulo 17

Frontend Web com CloudFront para Operação da API e Ingestão RAG

17.1 Escopo do capítulo

Este capítulo apresenta a camada de experiência do usuário da plataforma: uma interface web estática distribuída por *Amazon CloudFront*¹ com origem em bucket *Amazon S3* privado, destinada a dois fluxos principais: interação com a API assíncrona via `POST /query` e `GET /status/{request_id}` e ingestão controlada de documentos no pipeline *Retrieval-Augmented Generation* (RAG) via `POST /documentos`. O objetivo é fornecer uma superfície web de operação que possa ser utilizada por usuários de negócio, analistas de homologação e equipes técnicas sem necessidade de notebooks ou scripts auxiliares.

17.2 Objetivos de aprendizagem

Ao concluir este capítulo, o leitor será capaz de:

- Descrever a arquitetura de publicação de um frontend estático via CloudFront e bucket S3 privado com *Origin Access Control*;
- Implementar uma interface web para envio de consultas ao agente e acompanhamento do processamento assíncrono;
- Disponibilizar, na mesma interface, um fluxo de envio de documentos textuais para indexação no RAG;
- Avaliar limitações do backend atual para formatos binários como PDF e DOCX e propor a extensão arquitetural adequada.

¹Rede de distribuição de conteúdo (CDN) da AWS utilizada para publicar arquivos estáticos com baixa latência e alta disponibilidade.

17.3 Conceitos teóricos

17.3.1 Frontend estático como superfície de operação

Em arquiteturas bancárias modernas, a separação entre camada de apresentação e serviços de negócio reduz acoplamento, melhora a escalabilidade e simplifica a estratégia de publicação. Para o caso em estudo, um frontend estático atende adequadamente aos requisitos porque:

- o processamento de negócio ocorre integralmente no backend assíncrono (API Gateway + Lambda + SQS + DynamoDB);
- a UI atua apenas como orquestradora de chamadas HTTP, renderização de respostas e coleta de insumos do usuário;
- a camada estática pode ser publicada sem servidor dedicado, com custo operacional reduzido e baixa superfície de ataque.

17.3.2 CloudFront + S3 privado

A distribuição CloudFront é configurada para apontar para um bucket S3 privado que armazena os arquivos `index.html`, `styles.css`, `app.js` e `config.js`. O acesso direto ao bucket permanece bloqueado; apenas a distribuição CloudFront, por meio de *Origin Access Control* (OAC), possui permissão de leitura. Esse padrão é preferível ao uso de *website endpoint* público do S3, pois preserva o princípio de exposição mínima e centraliza a entrega HTTPS.

17.3.3 Chat assíncrono e ingestão RAG em uma única UI

A interface implementa dois fluxos distintos:

1. **Consulta assíncrona:** o usuário preenche pergunta, modo de operação e, opcionalmente, dados do cliente; a UI envia o payload para `/query`, recebe um `request_id` e executa *polling* em `/status/{request_id}` até o estado `COMPLETED`.
2. **Ingestão de documentos:** o usuário informa o tipo (documento ou entrevista), título e conteúdo, podendo carregar um arquivo textual local. O frontend converte o arquivo em texto e envia JSON para `/documentos`.

A limitação atual decorre do contrato do endpoint `/documentos`, que aceita apenas JSON com campo `texto`; não há suporte nativo a *multipart/form-data*, nem extração de conteúdo de formatos binários como PDF. Assim, a UI suporta apenas arquivos textuais e colagem manual de conteúdo.

17.4 Implementação orientada por passos

17.4.1 Passo 1 — Estruturar os assets do frontend

A aplicação web foi implementada sem *framework* JavaScript, com o objetivo de minimizar dependências e facilitar a publicação:

```
1 frontend/  
2   cloudfront_console/  
3     index.html  
4     styles.css  
5     app.js  
6     config.js
```

Listing 17.1: Estrutura dos arquivos estáticos do console web.

O arquivo `config.js` é sobrescrito no deploy Terraform com a URL real da API Gateway, eliminando a necessidade de compilação por ambiente.

17.4.2 Passo 2 — Publicar a UI em CloudFront

O módulo Terraform `cloudfront_frontend` cria:

- bucket S3 privado com versionamento e criptografia;
- OAC para leitura segura do bucket pela distribuição;
- distribuição CloudFront com `default_root_object = index.html`;
- objetos S3 contendo os assets do frontend e o `config.js` parametrizado.

```
1 module "cloudfront_frontend" {  
2   source      = "./modules/cloudfront_frontend"  
3   project_name = var.project_name  
4   environment  = var.environment  
5   api_endpoint = module.api_gateway.api_endpoint  
6 }  
7  
8 output "frontend_url" {  
9   value = "https://${module.cloudfront_frontend.  
10     distribution_domain_name}"  
}
```

Listing 17.2: Trecho do módulo Terraform do frontend CloudFront.

17.4.3 Passo 3 — Implementar o fluxo de consulta assíncrona

A interface web coleta pergunta, modo e dados do cliente e monta o payload compatível com o contrato OpenAPI da aplicação. Em seguida:

1. envia `POST /query`;
2. armazena o `request_id`;
3. inicia *polling* com *backoff* exponencial;
4. atualiza a UI com status intermediários e resultado final.

Esse padrão desacopla a experiência do usuário da duração real do pipeline RAG, compatibilizando a interface web com o limite de 29 segundos do API Gateway descrito no Capítulo 7.

17.4.4 Passo 4 — Implementar envio de documentos para RAG

O fluxo de ingestão aceita dois modos de entrada:

- **Upload de arquivo textual:** a UI lê o arquivo no navegador com `File.text()` e preenche o campo `texto`;
- **Colagem manual:** o usuário insere diretamente o conteúdo no *textarea*, permitindo testes e indexação rápida.

O payload enviado para a API segue o formato:

```
1 {  
2   "tipo": "documento",  
3   "titulo": "Politica de Investimentos 2026",  
4   "texto": "Clientes conservadores priorizam liquidez..."  
5 }
```

Listing 17.3: Payload de ingestão de documento no RAG.

Quando o tipo é *entrevista*, a UI pode enviar adicionalmente `cliente_id` ou `cluster_id`, alinhando a indexação com os modos *twin*, *persona* e *segmento*.

17.4.5 Passo 5 — Planejar evolução para PDF e DOCX

Para suportar documentos binários, recomenda-se introduzir uma etapa de extração antes da chamada a `/documentos`. Duas abordagens são plausíveis:

1. **Extração no backend:** novo endpoint de upload recebe o binário, armazena em S3 e aciona Lambda com parser (Apache Tika, PyMuPDF, python-docx ou Amazon Textract).
2. **Pré-processamento cliente:** alternativa limitada a alguns formatos e menos indicada para ambiente bancário, pois expõe a extração ao navegador.

Para o cenário bancário, a primeira abordagem é superior por motivos de rastreabilidade, auditoria e controle de segurança.

17.5 Plano de testes do capítulo

17.5.1 Teste funcional

- **TC-01:** Abrir a URL do CloudFront e verificar o carregamento de `index.html`, `styles.css` e `config.js` sem acesso direto público ao bucket S3.
- **TC-02:** Executar uma consulta em modo **segmento** e verificar que a UI exibe `request_id`, status intermediário e resposta final.
- **TC-03:** Submeter um documento textual e verificar resposta HTTP 200 com `chunks_indexados > 0`.
- **TC-04:** Tentar enviar um arquivo PDF e verificar que a UI informa explicitamente a limitação atual do backend.

17.5.2 Teste de validação em produção (AWS)

Foi executado um smoke test completo no ambiente publicado, com os seguintes endereços:

- **Frontend CloudFront:** <https://djxkmivr2akfz.cloudfront.net>;
- **API Gateway:** <https://3vqrahlmyg.execute-api.sa-east-1.amazonaws.com>.

Os resultados observados foram:

- **TVP-01 (ingestão entrevista):** `ok=true, chunks_indexados=1`;
- **TVP-02 (consulta persona):** `request_id: <ddbc5210-63fc-47bc-a274-283e60be84bc>`.
Estado final COMPLETED;
- **TVP-03 (consulta twin):** `request_id: <e5e348e7-d266-4ae9-9f26-eb0e659a56a7>`.
Estado final COMPLETED;

- **TVP-04 (correção de exibição da resposta):** Identificado e corrigido defeito no `app.js` em que o campo `data.result` era exibido diretamente como texto, resultando em `[object Object]`. A correção extrai `result.resposta` após *parse* JSON, exibindo o texto legível do agente no painel **Resposta** e os metadados (`cliente_id`, `cluster_id`, `confiabilidade`) no painel **Status**. Implantado em produção via `aws s3 sync` e invalidação do cache CloudFront em 02/05/2026.

Com isso, verifica-se a operação ponta a ponta da interface web em produção, incluindo ingestão RAG e consulta assíncrona com *polling* até a conclusão.

17.5.3 Teste de segurança

- **TS-01:** Confirmar que o bucket S3 do frontend possui *public access block* habilitado.
- **TS-02:** Verificar que apenas a distribuição CloudFront possui permissão de leitura nos objetos estáticos.
- **TS-03:** Validar cabeçalhos CORS da API para permitir consumo pelo frontend, sem abrir além do necessário.

17.6 Critérios de aceite

- TC-01 a TC-04 aprovados.
- TS-01 a TS-03 aprovados.
- URL `frontend_url` gerada pelo Terraform e acessível via HTTPS.
- Fluxos de consulta e ingestão operacionais em *dev*.
- Smoke test em produção aprovado para ingestão, *persona* e *twin* (TVP-01 a TVP-03).
- Resposta legível do agente exibida no painel **Resposta** via `result.resposta` (TVP-04 — defeito de exibição corrigido).

17.7 Espaço para expansão iterativa

- **v17.1** — Integrar autenticação com Amazon Cognito para restringir o acesso à interface web.
- **v17.2** — Adicionar histórico de consultas e visualização de status por usuário autenticado.

- **v17.3** — Suportar upload binário com pipeline de extração para PDF, DOCX e imagens escaneadas.
- **Lacuna** — Avaliar *WebSocket API* como alternativa ao *polling* HTTP para feedback de progresso em tempo real.

Apêndice A

Template de Capitulo

Este apendice serve como guia de expansao iterativa para novos capitulos.

A.1 Objetivos do capitulo

- Objetivo 1
- Objetivo 2

A.2 Implementacao pratica

1. Passo 1
2. Passo 2
3. Passo 3

A.3 Plano de testes

- Teste funcional
- Teste de performance
- Teste de qualidade de resposta

A.4 Criterios de aceite

- Criterio 1
- Criterio 2

Apêndice B

Glossário Matemático e Técnico

Este apêndice reúne definições formais dos principais termos matemáticos, estatísticos e de aprendizado de máquina empregados ao longo do livro. Cada entrada inclui notação, descrição e, quando pertinente, referência bibliográfica. Os termos aparecem em ordem alfabética.

B

Byte-Pair Encoding (BPE)

O *Byte-Pair Encoding* (BPE) é um algoritmo de tokenização por sub-palavras originalmente proposto para compressão de dados e posteriormente adaptado ao processamento de linguagem natural. O procedimento inicia com um vocabulário de caracteres individuais e, iterativamente, funde o par de símbolos adjacentes de maior frequência no corpus de treinamento, criando um novo símbolo composto. Após k fusões, obtém-se um vocabulário de tamanho $|\mathcal{V}| = |\mathcal{V}_0| + k$, em que $\mathcal{V}_0$ é o vocabulário inicial de caracteres.

1. Inicializar $\mathcal{V}$ com todos os caracteres únicos do corpus.
2. Contar a frequência de todos os pares de símbolos adjacentes.
3. Fundir o par mais frequente (a, b) em um novo símbolo ab .
4. Repetir os passos 2–3 até atingir $|\mathcal{V}|$ desejado.

O BPE oferece equilíbrio entre vocabulários de caracteres (sem palavras desconhecidas, mas sequências longas) e vocabulários de palavras inteiras (sequências curtas, mas alta taxa de *out-of-vocabulary*).

E

Embedding (vetor de representação)

Um *embedding* é uma função $f : \mathcal{X} \rightarrow \mathbb{R}^d$ que mapeia elementos de um espaço discreto (tokens, frases, documentos) para vetores densos em $\mathbb{R}^d$. No contexto de LLMs, a camada de *embedding* de entrada $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ associa cada token $t \in \mathcal{V}$ a um vetor $\mathbf{e}_t \in \mathbb{R}^d$. A dimensão d varia tipicamente entre 768 (modelos base) e 4096 (modelos de grande porte). Geometricamente, tokens semanticamente próximos tendem a ocupar regiões próximas no espaço vetorial, propriedade explorada pela busca por similaridade de cosseno:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

L

Logits

O termo *logits* designa os valores reais não normalizados $\mathbf{z} \in \mathbb{R}^{|\mathcal{V}|}$ produzidos pela última camada linear do modelo antes da aplicação da função de ativação. Formalmente, dado o vetor de estado oculto $\mathbf{h}$ da última camada do *Transformer*, os logits são calculados como:

$$\mathbf{z} = \mathbf{W}_{\text{lm}} \mathbf{h} + \mathbf{b},$$

em que $\mathbf{W}_{\text{lm}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ é a matriz de projeção do modelo de linguagem e $\mathbf{b}$ é o viés. Os logits são convertidos em probabilidades pela função *softmax* (ver Seção B).

S

SentencePiece

O *SentencePiece* é uma biblioteca de tokenização por sub-palavras independente de idioma desenvolvida pelo Google. Ao contrário do BPE clássico (ver Seção B), o *SentencePiece* opera diretamente sobre o fluxo de caracteres Unicode sem pré-segmentação por espaços, o que o torna aplicável a idiomas sem delimitadores explícitos de palavras (japonês, chinês, tailandês). O algoritmo suporta dois modelos: BPE e *unigram language model*, que seleciona o vocabulário minimizando a perda de verossimilhança do corpus sob um modelo de linguagem de unigrama.

Softmax

A função *softmax* $\sigma : \mathbb{R}^n \rightarrow \Delta^{n-1}$ converte um vetor de logits $\mathbf{z} = (z_1, \dots, z_n)$ em uma distribuição de probabilidade sobre n classes:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad i = 1, \dots, n.$$

No contexto da geração de texto, $n = |\mathcal{V}|$ e cada $\sigma(\mathbf{z})_i$ representa a probabilidade do token i ser o próximo token gerado. A *temperatura* $\tau > 0$ escala os logits antes da *softmax*:

$$\sigma(\mathbf{z}/\tau)_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}.$$

Para $\tau \rightarrow 0$, a distribuição concentra-se no token de maior logit (*greedy decoding*); para $\tau > 1$, a distribuição torna-se mais uniforme, aumentando a diversidade das saídas.

W

WordPiece

O *WordPiece* é um algoritmo de tokenização por sub-palavras desenvolvido pelo Google e utilizado em modelos como BERT. Em vez de fundir o par de maior frequência absoluta (como no BPE), o *WordPiece* seleciona o par (a, b) que maximiza o ganho de verossimilhança do modelo de linguagem ao substituir a sequência ab por um único token, definido como:

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \cdot \text{freq}(b)}.$$

Essa formulação favorece fusões de pares cujos componentes individuais são relativamente raros, produzindo um vocabulário de sub-palavras com maior coerência linguística.

Sobre os Autores

Texto reservado para biografia dos autores.

Referências Bibliográficas

- 1 WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, v. 35, p. 24824–24837, 2022. Disponível em: <<https://arxiv.org/abs/2201.11903>>.
- 2 VASWANI, A. et al. Attention is all you need. *Advances in Neural Information Processing Systems*, v. 30, 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>.
- 3 WEI, J. et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. Disponível em: <<https://arxiv.org/abs/2206.07682>>.
- 4 OpenAI. *tiktoken: Fast BPE tokeniser for use with OpenAI models*. 2023. Acessado em: maio de 2026. Disponível em: <<https://github.com/openai/tiktoken>>.
- 5 Anthropic. *Prompt Engineering Guide for Claude*. 2024. Acessado em: maio de 2026. Disponível em: <<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>>.
- 6 LangChain AI. *LangGraph: Build stateful, multi-actor applications with LLMs*. 2024. Acessado em: maio de 2026. Disponível em: <<https://github.com/langchain-ai/langgraph>>.
- 7 CHASE, H. *LangChain: Building applications with LLMs through composability*. 2022. Acessado em: maio de 2026. Disponível em: <<https://github.com/langchain-ai/langchain>>.
- 8 YAO, S. et al. ReAct: Synergizing reasoning and acting in language models. 2023. Disponível em: <<https://arxiv.org/abs/2210.03629>>.
- 9 JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, v. 7, n. 3, p. 535–547, 2019.

Índice Remissivo

- action (ReAct), 14
- add_messages, 15
- agente (IA), **14**
- Amazon Bedrock, **3**
- ambiente virtual (Python), **10**
- amostragem por núcleo, *veja* top-p
- análise estática de segurança, 64
- API Gateway, **11**, **30**, 57, 70
- arquitetura
 - Transformer, 2
- arquitetura assíncrona, **30**
- AsyncPostgresSaver, **20**
- atenção
 - multi-head, 2
 - self-attention, 2
- AWS, 8, 56
- AWS Access Analyzer, 64
- AWS Well-Architected Framework, 57
- BACEN, **63**
- Banco Central do Brasil, *veja* BACEN
- bandit (segurança), 64
- Bedrock, *veja* Amazon Bedrock
- BPE, *veja* Byte-Pair Encoding
- Byte-Pair Encoding, **2**, **78**
- ChatBedrock, **16**
- checkov, 64
- checkpointer, 15
- chunking, **27**
- CI/CD, **56**
- CloudFront, **71**
- CloudTrail, **63**
- clusterizacao, **35**
- Cognito, **76**
- config.js, **73**
- conta AWS separada, 56
- context window, *veja* janela de contexto
- DAST, **70**
- dead-letter queue, 34
- decoder, 2
- decodificação, 3
- dev (ambiente), **56**
- digital twin, **35**
- drift detection, 62
- DynamoDB, **11**, **30**, 57
- EKS (AWS), 62
- embedding, **79**
- environment protection rules, 60
- FAISS, **14**, 18
- ferramenta (agente), **16**
- few-shot, **6**
- fine-tuning, **6**
- fire-and-poll, **30**
- Git Workflow, **56**
- GitHub Actions, **56**
- GitLab CI, 56
- GitOps, 62
- grafo dirigido, 15
- GraphRAG, **41**
- homol (ambiente), **56**
- IAM, 3, 10, 57
 - menor privilégio, 64

- Identity and Access Management, *veja* IAM, *veja* IAM
- indexacao (RAG), **25**
- inferência, **9**
- infraestrutura como código, *veja* Terraform
- integração contínua, **56**
- janela de contexto, **3**
- K-Means, **35**
- Lambda (AWS), **11**, **57**
- LangChain, **14**
- LangGraph, **14**
- logits, **3**, **79**
- memória
 - agentes, **14**
- MemorySaver, **15**
- menor privilégio, **63**
- MessagesState, **15**
- Neptune (AWS), **11**, **41**, **57**
- next-token prediction, **2**
- observation (ReAct), **14**
- OIDC, **61**
- Open Policy Agent, **70**
- OpenCypher, **41**
- OpenID Connect, *veja* OIDC
- OpenSearch, **11**, **25**, **57**
- Origin Access Control, **71**
- OWASP Top 10, **64**
- padronizacao (ML), **36**
- path ranking, **45**
- persona (IA), **35**
- PLN, *veja* processamento de linguagem natural
- policy-as-code, **70**
- processamento de linguagem natural, **2**
- prod (ambiente), **56**
- prompt
 - few-shot, **6**
 - system prompt, **5**
- pull request, **57**
- Python, **8**
- quality gate, **63**
- RAG, **25**, **71**
- ReAct, **14**
- recuperação semântica, **18**
- recuperacao semantica, **25**
- redes recorrentes, **2**
- reprodutibilidade, **9**
- request_id, **72**
- reranking, **29**
- S3 (AWS), **58**, **71**
- SBOM, **70**
- segmentação subpalavra, **2**
- segurança (esteira), **63**
- self-attention, **2**
- SentencePiece, **2**, **79**
- Simple Queue Service, *veja* SQS
- SNS (AWS), **62**
- softmax, **2**, **3**, **80**
- SQS, **11**, **30**, **57**
- StandardScaler, **36**
- state backend, **56**
- state file (Terraform), **59**
- StateGraph, **16**
- superfície de ataque, **61**
- temperatura (LLM), **3**
- Terraform, **8**, **56**
 - workspace, **56**
- testes de integração, **65**
- Textextract, **75**
- tfsec, **64**
- thought (ReAct), **14**
- thread_id, **15**

token, 2

ToolNode, **16**

top-k, **3**

top-p, **3**

Transformer, **2**

TypedDict, 15

versionamento semântico, **57**

vetor semântico, 14

Virtual Private Cloud, *veja* VPC

VPC, **11**

WordPiece, **80**